

Aula 15

DataPrev (Perfil 3: Desenvolvimento de Software) Desenvolvimento de Software - 2026 (Pós-Edital)

Autor:
Vinicius Borges

09 de Julho de 2026

Índice

1) Microserviços SAGA - Teoria	3
2) Microserviços SAGA - Questões comentadas	15
3) Microserviços SAGA - Lista de questões	23
4) Microserviços - Circuit Breaker - Teoria	28
5) Microserviços - Circuit Breaker - Questões comentadas	40
6) Microserviços - Circuit Breaker - Lista de questões	47
7) Microserviços e BFF - Teoria	52
8) Microserviços e BFF - Questões comentadas	65
9) Microserviços e BFF - Lista de questões	72
10) API gateway - Teoria	77
11) API gateway - Questões comentadas	81
12) API gateway - Lista de questões	90

MICROSSERVIÇOS E O PADRÃO SAGA

Transações distribuídas em microsserviços

Olá, pessoal! Vamos iniciar nosso estudo com uma pergunta provocativa: quando um e-commerce confirma um pedido, ele precisa cobrar o cartão, reservar o estoque e disparar a entrega. Em um monolito tradicional, essas três operações cabem dentro de uma única transação ACID do banco; se qualquer uma falha, o banco faz o **rollback** e a aplicação se mantém íntegra. O que acontece quando cada uma dessas responsabilidades vive em um serviço próprio, com banco próprio? É aí que entra a discussão de transações distribuídas, terreno em que o padrão SAGA brilha mais alto (perdão, em que o padrão SAGA é a resposta mais difundida).

Em uma arquitetura de microsserviços bem desenhada, cada serviço tem seu próprio banco e ninguém mais escreve nele. Esse princípio é chamado de **database per service**, e existe por uma razão prática: permite que cada equipe evolua o esquema do seu serviço sem coordenação com as outras. O efeito colateral, porém, é que a transação ACID do SGBD vale apenas dentro de um serviço; ela não atravessa as fronteiras da rede.

Por que ACID local não basta

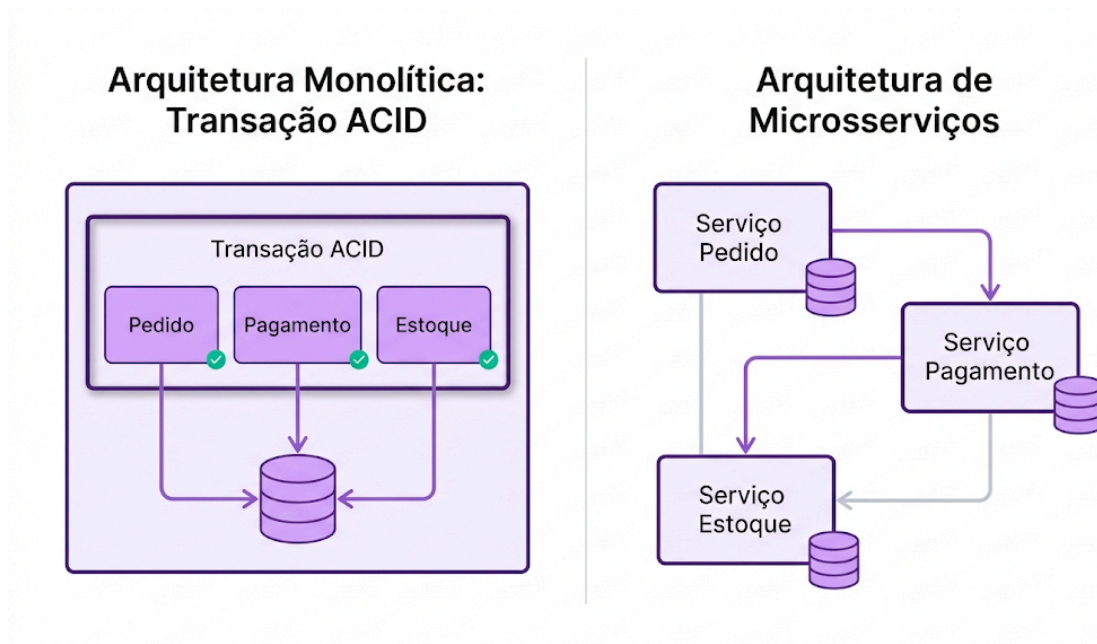
Considere o fluxo de pedido: o serviço Pedido cria o registro, o serviço Pagamento cobra o cartão e o serviço Estoque reserva os itens. Cada um faz commit no seu banco. Se o Pagamento der erro depois que Pedido já gravou, o registro de Pedido fica órfão, sem cobrança nem reserva. Não há um BEGIN/COMMIT comum aos três bancos: cada serviço só conhece o próprio.

A reação clássica do mundo SQL para esse cenário é o **Two-Phase Commit** (2PC), padronizado pela X/Open XA. Nele, um coordenador global pergunta a cada participante: "você consegue commitar?". Se todos respondem sim na fase prepare, o coordenador envia o commit final na fase commit. Se algum responde não, ele envia rollback a todos. Em papel, é elegante; na prática distribuída, é lento e bloqueante.

Limitações do 2PC em escala

Três fatores condenam o 2PC em microsserviços. Primeiro, ele bloqueia recursos durante a fase de preparação; enquanto um participante mantém um lock à espera do commit final, qualquer outro pedido que toque a mesma linha fica esperando. Segundo, ele exige que todos os bancos suportem o protocolo XA, o que exclui boa parte dos bancos NoSQL e nem todo SGBD relacional oferece a implementação madura. Terceiro, em uma falha do coordenador entre prepare e commit, os participantes ficam em um estado indeterminado ("in-doubt"), com locks pendurados até alguém intervir manualmente.

O cenário se agrava com o número de serviços. Cada participante adicional multiplica o tempo total do 2PC, e a probabilidade de pelo menos um deles estar indisponível cresce junto. Em sistemas com SLA alto e bancos diversos, o custo simplesmente não fecha.



Observe que, no monolito (à esquerda), as três operações cabem dentro de um único BEGIN/COMMIT. Em microserviços (à direita), cada banco é uma fronteira transacional; precisamos de outro mecanismo para coordenar as três etapas.

Fonte: Prof. Vinicius Borges

Para a prova:

- Cada microserviço tem seu próprio banco (database per service); a transação ACID só vale localmente.
- O 2PC bloqueia recursos na fase de preparação e exige suporte XA em todos os participantes.
- Em arquiteturas distribuídas com muitos serviços, o 2PC reduz disponibilidade e dificulta escalabilidade.

HORA DE PRATICAR!



(PROF. VINICIUS BORGES / INÉDITA - 2026)

Acerca do gerenciamento de transações em microsserviços, julgue o item: o protocolo Two-Phase Commit (2PC) tende a ser pouco adequado em arquiteturas com muitos serviços porque bloqueia recursos durante a fase de preparação e demanda suporte XA em todos os participantes.

Comentários:

Conforme estudamos nesta seção, o 2PC bloqueia recursos enquanto aguarda o **commit** final do coordenador, o que reduz a disponibilidade à medida que o número de participantes cresce. Além disso, exige que todos os bancos suportem o protocolo XA, condição que nem sempre é satisfeita em ambientes poliglotos. Por isso, em microsserviços, costuma-se preferir SAGA.

Gabarito: Certo

O padrão SAGA: ideia central

O padrão SAGA propõe uma resposta pragmática para a transação distribuída: em vez de tentar coordenar um commit global, decomparamos a transação em uma **sequência de transações locais**, cada uma rodando no banco do seu próprio serviço. Cada etapa T_i , ao terminar, dispara a próxima. Se alguma T_i falhar, executamos transações compensatórias $C(i-1)$, $C(i-2)$, ..., C_1 , que desfazem logicamente os efeitos das etapas anteriores.

A formulação original é de 1987, no contexto de transações de longa duração em bancos. O padrão ressurgiu com força nos anos 2010 ao se mostrar perfeitamente alinhado ao modelo de microsserviços, em que cada etapa naturalmente vive em um serviço diferente.

Anatomia de uma SAGA

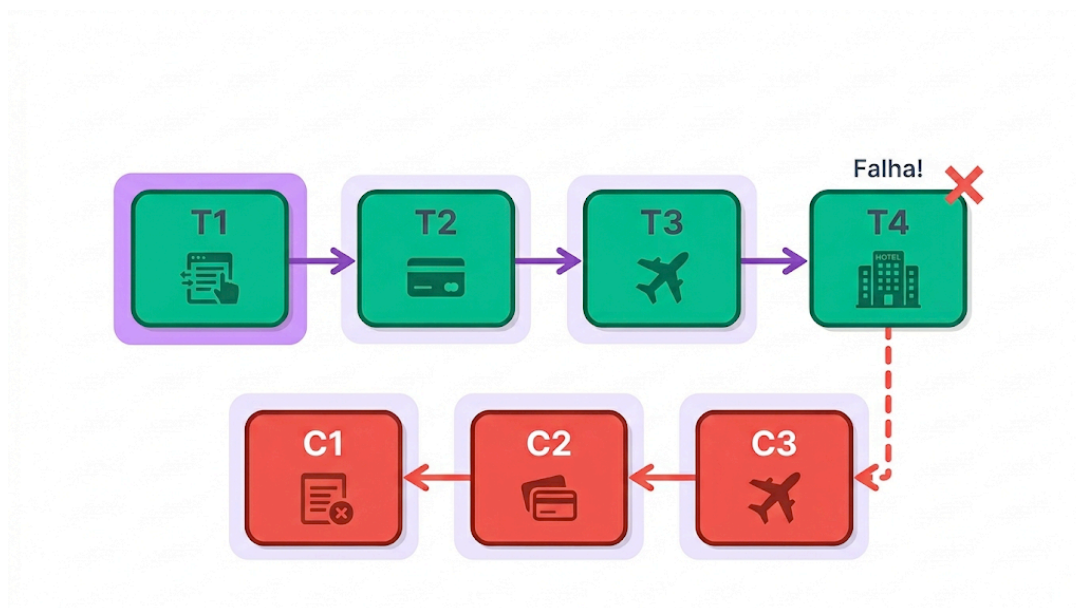
Uma SAGA típica tem três peças: as transações locais T_i (cada uma executa um pedaço do trabalho de negócio com ACID local), as transações compensatórias C_i (cada uma desfaz logicamente a respectiva T_i) e um mecanismo de coordenação que liga uma etapa à seguinte. O mecanismo pode ser centralizado (orquestração) ou descentralizado (coreografia); veremos os dois adiante.

Um ponto crítico é entender que a transação compensatória **não é um rollback técnico**. Como T_1 já foi comitada no banco do serviço A, o SGBD não tem mais o que reverter; o que C_1 faz é executar uma nova transação que desfaz o efeito de negócio de T_1 . Se T_1 foi "cobrar R\$ 100", C_1 é "estornar R\$ 100". O extrato registra os dois lançamentos, e o saldo volta ao ponto anterior do ponto de vista do cliente.

Consistência eventual

Como cada T_i efetua commit local de forma independente, a SAGA passa por estados intermediários em que o sistema não está globalmente consistente. Esse é o trade-off central: trocamos a consistência forte do 2PC pela **consistência eventual**. A consistência se restabelece ao final da SAGA, quando todas as etapas concluem com sucesso, ou após as compensações, no caso de falha.

Esse modelo não é exótico nem inferior; é uma escolha consciente alinhada ao teorema CAP. Em sistemas distribuídos sujeitos a partições de rede, o SAGA prioriza disponibilidade (A) e tolerância a partições (P), aceitando que a consistência (C) seja eventual. Para a maioria dos casos de negócio (pedido, reserva, cadastro), essa janela curta de inconsistência é perfeitamente tolerável.



Repare na ordem das compensações: se T4 falha, executa-se C3, depois C2 e depois C1, sempre no caminho inverso da execução.

Fonte: Prof. Vinicius Borges

Conceito	Definição	Exemplo prático
Transação local T_i	Operação ACID dentro do banco de um serviço	Reservar item no estoque
Compensação C_i	Transação que desfaz logicamente o efeito de T_i	Devolver item ao estoque
Consistência eventual	Sistema globalmente consistente ao fim da SAGA	Pedido confirmado depois de Pagamento e Estoque

Para a prova:

- SAGA = sequência de transações locais com compensações associadas.
- Compensação é uma nova transação que desfaz logicamente o efeito, não um ROLLBACK técnico do SGBD.
- A ordem das compensações é estritamente inversa à da execução.
- O modelo de consistência adotado é a consistência eventual.

HORA DE PRATICAR!



(PROF. VINICIUS BORGES / INÉDITA - 2026)

No padrão SAGA, se uma SAGA composta por T1, T2 e T3 falha em T3, a recuperação se dá por meio:

- a) do envio de um comando ROLLBACK ao banco de cada serviço, revertendo as escritas em T1 e T2.
- b) da execução das compensações C2 e C1, nessa ordem, que desfazem logicamente os efeitos das etapas anteriores.
- c) da reexecução automática de T1, T2 e T3 até que todas obtenham sucesso simultâneo.

Comentários:

Conforme vimos nesta seção, T1 e T2 já foram comitadas em seus bancos, e o SGBD não as reverte automaticamente. O caminho é executar **C2 e C1, nessa ordem**, que são novas transações com efeito inverso ao de T2 e T1.

Gabarito: Letra B

Variações: orquestração e coreografia

O padrão SAGA admite duas formas principais de implementação, que se diferenciam por **quem decide a próxima etapa**. Na coreografia, ninguém decide centralmente: cada serviço reage a eventos publicados pelos outros. Na orquestração, um componente dedicado, o orquestrador, envia comandos e recebe respostas, comandando o fluxo. As duas são válidas, e a escolha depende do perfil do problema.

Coreografia: SAGA por eventos

Na coreografia, os serviços se comunicam por um broker de mensagens (RabbitMQ, Kafka, AWS SNS/SQS) que distribui eventos para quem se inscreveu nos tópicos relevantes. O fluxo de pedido fica assim: o serviço Pedido publica **PedidoCriado**; Pagamento, inscrito nesse tópico, processa a cobrança e publica **PagamentoAprovado**; Estoque, inscrito nesse outro tópico, reserva o item e publica **ReservaConfirmada**. A SAGA caminha sem coordenador.

Se Estoque falha, ele publica **ReservaFalhou**. Pagamento, que também escuta esse evento, dispara o estorno; Pedido marca o registro como cancelado. As compensações fluem pelo mesmo mecanismo: eventos. O baixo acoplamento é o atrativo: cada serviço só sabe dos eventos que consome e produz, sem dependência direta de qualquer outro.

O preço dessa autonomia é a dificuldade de raciocinar sobre o fluxo global. Em um fluxo curto, com 3 ou 4 etapas, isso é gerenciável. Quando a SAGA cresce e ganha ramificações, descobrir o estado global se torna doloroso, pois a lógica fica espalhada pelos handlers de cada serviço. Há também o risco de ciclos involuntários, em que um evento de compensação dispara outra cadeia que não estava prevista.

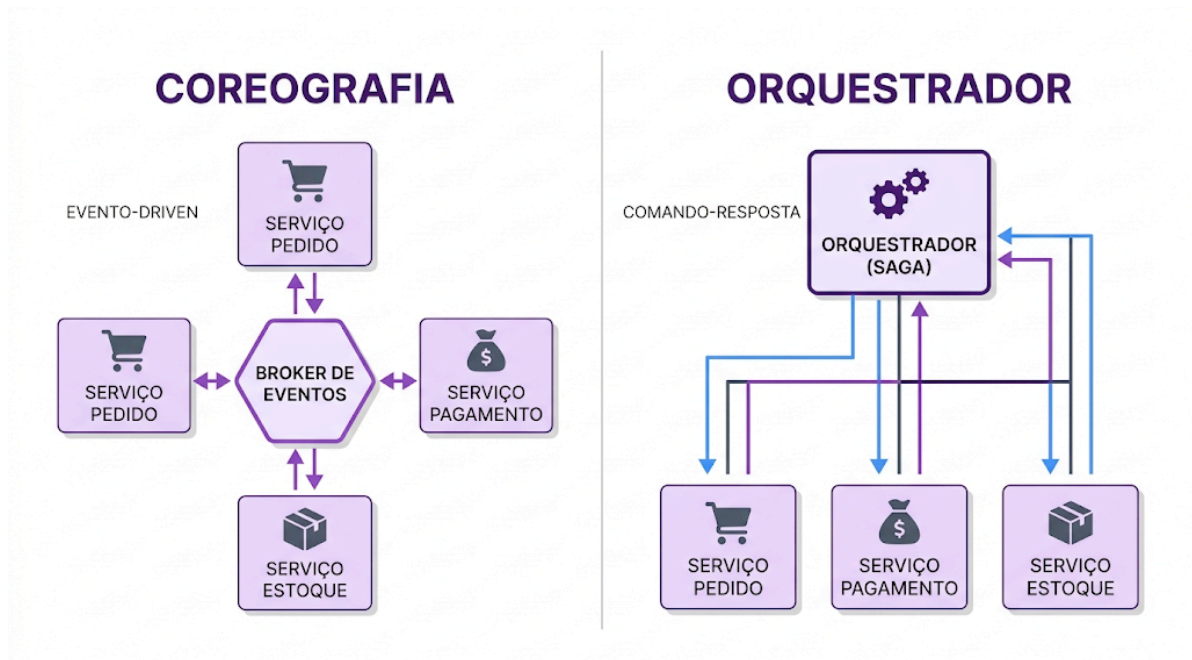
Orquestração: SAGA por comandos

Na orquestração, um componente central, o **orquestrador** (também chamado SAGA Execution Coordinator), envia comandos aos serviços e aguarda respostas para decidir o próximo passo. O fluxo de pedido seria: o orquestrador envia **ReservarEstoque** ao Estoque; recebe **EstoqueReservado**; envia **CobrarCartao** ao Pagamento; recebe **PagamentoAprovado**; envia **ConfirmarPedido** ao Pedido; conclui. Em qualquer falha, o próprio orquestrador dispara as compensações na ordem inversa.

Como o estado da SAGA está concentrado em um lugar, fica natural responder a perguntas operacionais como "em que etapa o pedido 12345 está?" ou "quantas SAGAs estão pendentes há mais de 5 minutos?". Esse ganho de observabilidade é o que faz a orquestração ser preferida em fluxos complexos, com muitas ramificações condicionais.

O custo é a centralização: o orquestrador vira um componente crítico, sujeito a se tornar gargalo de desempenho ou ponto único de falha. Em geral, mitiga-se isso com tecnologias dedicadas,

como **Camunda**, **Temporal** ou **AWS Step Functions**, que oferecem persistência do estado, retries, timeouts e dashboards prontos.



Compare a topologia: na coreografia, os serviços conversam pelo broker em uma malha; na orquestração, o orquestrador é o hub e os serviços são raios.

Fonte: Prof. Vinicius Borges

Critério	Coreografia	Orquestração
Coordenação	Distribuída (eventos)	Centralizada (comandos)
Acoplamento	Baixo (serviços se ignoram)	Médio (todos conhecem o orquestrador)
Observabilidade	Difícil em fluxos longos	Boa, estado no orquestrador
Adequação	Fluxos curtos e simples	Fluxos longos com ramificações
Risco principal	Lógica espalhada, ciclos involuntários	Orquestrador como gargalo

Para a prova:

- Coreografia: serviços reagem a eventos via broker; sem coordenador central.
- Orquestração: um componente central envia comandos e decide a próxima etapa.
- Coreografia favorece baixo acoplamento; orquestração favorece observabilidade.

- Em qualquer das duas variações, as compensações são acionadas na ordem inversa.

HORA DE PRATICAR!



(PROF. VINICIUS BORGES / INÉDITA - 2026)

Sobre as variações do padrão SAGA, assinale a alternativa correta.

- a) Na coreografia, um orquestrador central envia comandos aos serviços e decide a próxima etapa.
- b) Na orquestração, os serviços apenas reagem a eventos publicados em um broker, sem coordenação centralizada.
- c) A coreografia favorece baixo acoplamento entre serviços, enquanto a orquestração favorece a observabilidade do fluxo.
- d) Em ambas as variações, as compensações são executadas na mesma ordem da execução original.

Comentários:

Conforme estudamos, a coreografia tem como atrativo o **baixo acoplamento** entre os serviços (eles só sabem dos eventos), enquanto a orquestração concentra o estado em um componente, o que melhora a **observabilidade**.

Gabarito: Letra C

Cuidados práticos: idempotência, retries e observabilidade

Implementar SAGA na vida real exige atenção a três temas que extrapolam o desenho conceitual: idempotência dos consumidores, política de retries com timeouts e observabilidade do fluxo. Sem esses três, o padrão pode até funcionar no caminho feliz, mas vai colapsar na primeira falha de rede.

Idempotência: a fundação prática

Em qualquer SAGA baseada em mensageria, é regra que uma mesma mensagem possa ser entregue mais de uma vez. Os brokers populares (Kafka, RabbitMQ, SQS) implementam, no caso geral, semântica **at-least-once**: eles garantem que a mensagem chega, mas podem entregar duplicada em falhas de rede ou rebalanceamentos. A entrega **exactly-once** de ponta a ponta existe em configurações específicas, mas costuma ser cara e frágil; a prática consagrada é cooperação do consumidor por meio de idempotência.

Tornar um handler idempotente significa fazer com que processar a mesma mensagem N vezes produza o mesmo efeito de processá-la uma única vez. A técnica clássica é dar a cada comando ou evento um identificador único (UUID, ou correlation-id que carrega o id da SAGA + número da etapa) e o serviço manter uma tabela de mensagens já processadas. No início do handler, consulta-se essa tabela: se a mensagem já consta, retorna sucesso silenciosamente; senão, processa e registra. Essa tabela pode ser combinada com a transação local para garantir atomicidade entre o efeito de negócio e o registro da idempotência.

Retries, timeouts e dead-letter queues

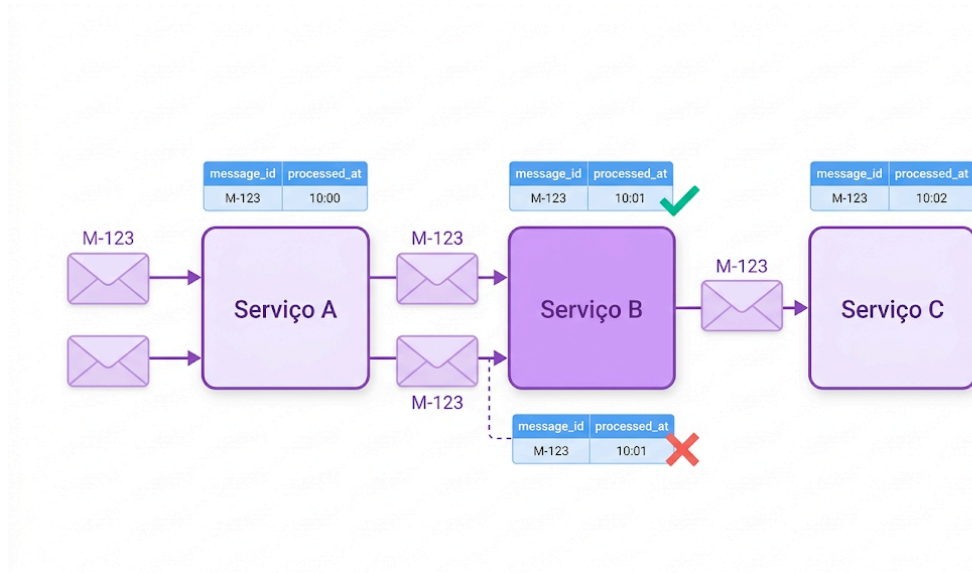
Quando uma etapa falha por motivo transitório (rede instável, banco temporariamente indisponível), faz sentido tentar de novo antes de disparar as compensações. A política usual envolve retries com **backoff exponencial**, limitado por um número máximo de tentativas ou por um deadline. Esgotado o retry, a SAGA decide entre compensar ou parar para intervenção humana, dependendo da criticidade.

Mensagens que reiteradamente falham vão para uma **dead-letter queue**, uma fila especial em que ficam expostas para análise. O dead-lettering protege a fila principal contra mensagens venenosas e gera visibilidade operacional sobre o que está dando errado.

Observabilidade e correlation-id

Para acompanhar o fluxo de uma SAGA pelos vários serviços, é praticamente obrigatório propagar um **correlation-id** em todas as mensagens, logs e traces. Com esse id, é possível filtrar em uma ferramenta como Jaeger, Zipkin ou Datadog APM tudo o que aconteceu com aquele pedido específico, em qualquer dos serviços envolvidos. Sem correlation-id, depurar uma SAGA que travou parece detetive sem lanterna.

Na orquestração, o próprio orquestrador já mantém o estado da SAGA persistido (em geral em um banco dedicado), o que oferece um nível razoável de observabilidade por padrão. Na coreografia, o correlation-id é a única forma prática de reconstruir o fluxo a partir dos logs distribuídos.



Note a tabela de mensagens processadas: a primeira entrega registra o id; a segunda, ao ver o id já presente, retorna sem efeito colateral. Esse é o mecanismo de idempotência em ação.

Fonte: Prof. Vinicius Borges

Para a prova:

- Brokers de mensageria normalmente oferecem at-least-once; idempotência é responsabilidade do consumidor.
- A idempotência usa identificadores únicos por mensagem e tabela de processados.
- Retries com backoff exponencial + dead-letter queue são padrão para falhas transitórias.
- Correlation-id propagado é essencial para observabilidade do fluxo da SAGA.

HORA DE PRATICAR!



(PROF. VINICIUS BORGES / INÉDITA - 2026)

Acerca de aspectos práticos de implementação do padrão SAGA, julgue o item: como os principais brokers oferecem semântica at-least-once de entrega, é prática recomendada tornar idempotente o handler de cada serviço, identificando mensagens já processadas a partir de um id único.

Comentários:

Realmente, esse é o caminho consagrado. Como vimos nesta seção, é mais barato e robusto fazer o consumidor lidar com duplicidades do que tentar garantir **exactly-once** de ponta a ponta no broker. A tabela de mensagens processadas, combinada à transação local, dá ao serviço resiliência frente a retransmissões.

Gabarito: Certo

QUESTÕES COMENTADAS

HORA DE PRATICAR!



1. (PROF. VINICIUS BORGES / INÉDITA - 2026)

O padrão SAGA, no contexto de microsserviços, propõe uma forma de coordenar transações que abrangem múltiplos serviços. Sobre o padrão SAGA, é correto afirmar que:

- a) divide uma transação distribuída em uma sequência de transações locais, cada uma com uma transação compensatória associada para o caso de a sequência precisar ser desfeita.
- b) impõe bloqueio distribuído entre os serviços envolvidos, no estilo two-phase locking, para garantir o isolamento da transação global.
- c) exige que todos os serviços participantes compartilhem o mesmo banco de dados relacional, de modo a preservar as propriedades ACID.
- d) substitui as transações locais por uma única transação ACID global, coordenada por um banco intermediário.
- e) elimina a necessidade de tratamento de falhas, pois delega ao gateway HTTP o rollback automático em caso de erro.

Comentários:

O padrão SAGA, descrito originalmente por Garcia-Molina e Salem em 1987 e ressignificado no contexto de microsserviços, decompõe uma transação de longa duração em uma sequência de transações locais $T_1, T_2, \dots, T_n$. Cada T_i executa no banco do seu próprio serviço (ACID local) e tem associada uma transação compensatória C_i , que desfaz logicamente o efeito de T_i caso a SAGA precise ser abortada. Não há bloqueio distribuído, não há banco compartilhado e não existe uma transação ACID global; o modelo aceita janelas de inconsistência intermediária em troca de disponibilidade.

Gabarito: Letra A

2. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Acerca do padrão SAGA aplicado a microsserviços, julgue o item a seguir.

Por dividir a transação distribuída em transações locais coordenadas a posteriori, o padrão SAGA adota o modelo de consistência eventual, em vez da consistência forte característica do protocolo Two-Phase Commit.

Comentários:

Realmente, o padrão SAGA assume um modelo de consistência eventual, e essa é a contrapartida natural ao abrir mão das garantias do 2PC. Como cada transação local efetua commit de forma independente, existe uma janela em que o sistema observa um estado intermediário, por exemplo, pagamento confirmado e estoque ainda não reservado. A consistência se restabelece ao fim da SAGA, quando todas as etapas concluem com sucesso, ou após a execução das compensações, no caso de falha. Essa flexibilidade é o que torna o SAGA adequado a sistemas distribuídos com alta disponibilidade.

Gabarito: Certo

3. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Em uma arquitetura de microsserviços que adota o padrão SAGA na variação por coreografia, qual característica é essencial dessa abordagem?

- a) A existência de um serviço orquestrador central, que envia comandos sequenciais aos demais participantes da transação.
- b) Cada serviço participante reage a eventos publicados por outros serviços, sem coordenação centralizada do fluxo.
- c) A presença de um Transaction Manager XA, responsável por enviar prepare, commit e rollback aos recursos transacionais.
- d) O uso de chamadas síncronas REST como mecanismo principal de comunicação entre os participantes da SAGA.
- e) A serialização das transações locais por meio de locks compartilhados em um banco distribuído único.

Comentários:

A coreografia, conforme estudaremos na teoria, é a variação do SAGA em que cada serviço publica e consome eventos por meio de um broker de mensagens (RabbitMQ, Kafka, etc.). Não há um coordenador central: o serviço A publica um evento, o serviço B reage e publica outro evento, e assim por diante. O fluxo emerge da composição dos handlers. Por isso, a coreografia é elogiada pelo baixo acoplamento, mas pode dificultar o entendimento global do fluxo quando há muitas ramificações. As demais alternativas descrevem padrões diferentes, como orquestração, 2PC e comunicação síncrona REST.

4. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Acerca da variação por orquestração do padrão SAGA, julgue o item a seguir.

Nessa variação, existe um componente central, o orquestrador, que envia comandos aos serviços participantes e decide a próxima etapa com base nas respostas recebidas, o que facilita rastrear o estado global da SAGA.

Comentários:

A orquestração introduz um componente dedicado (em geral implementado com motores como Camunda, Temporal ou um simples state machine caseiro) que executa o papel de maestro: emite comandos como ReservarEstoque, CobrarCartao, ConfirmarPedido e aguarda as respostas para decidir o próximo passo. Como o estado da SAGA está concentrado em um único lugar, fica natural consultar em que etapa cada transação está, o que ajuda em observabilidade e em fluxos com muitas ramificações. O custo é a centralização e o risco de o orquestrador se tornar gargalo arquitetural.

Gabarito: Certo

5. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Considere uma SAGA composta pelas transações locais T1, T2 e T3, executadas nessa ordem. Se T3 falhar, o padrão prescreve a execução de transações compensatórias para reverter as etapas anteriores. Sobre essas compensações, assinale a alternativa correta.

- a) Consistem em um rollback transacional do banco, idêntico ao comando ROLLBACK do SQL, aplicado em cada serviço participante.
- b) Executam apenas sobre T3, pois somente a etapa que falhou precisa ser revertida.
- c) São executadas na ordem inversa, primeiro a compensação de T2 e depois a de T1, e devem desfazer logicamente os efeitos já confirmados.
- d) Bloqueiam os recursos das transações T1 e T2 até que o orquestrador decida abortar definitivamente a SAGA.
- e) São opcionais, pois o broker de mensagens reverte automaticamente os eventos já publicados pelos serviços.

Comentários:

As transações compensatórias formam o coração da estratégia de recuperação do SAGA. Como T1 e T2 já foram comitadas em seus bancos locais, não é possível dar um ROLLBACK técnico nelas; o que se faz é executar novas transações C2 e C1, que desfazem o efeito de negócio, por exemplo, EstornarPagamento como compensação de CobrarPagamento. A ordem é estritamente inversa, justamente porque uma compensação pode depender do estado deixado pela compensação seguinte. As alternativas A, B, D e E confundem compensação com rollback, com bloqueio ou com reversão automática do broker, o que não corresponde ao padrão.

Gabarito: Letra C

6. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Acerca das transações compensatórias no padrão SAGA, julgue o item a seguir.

A transação compensatória executa um rollback transacional sobre o banco do serviço, revertendo bit a bit o estado anterior à transação local que se deseja desfazer.

Comentários:

Cuidado, pessoal! Há uma troca sutil de conceito aqui. A compensação no SAGA não é um rollback técnico; é uma nova transação que desfaz logicamente o efeito da anterior. Pense em uma compra: se a etapa CobrarCartao já foi confirmada, não é possível reverter o lançamento original do cartão (a operação já existe no extrato). O que se faz é registrar uma operação inversa, um estorno, que mantém o histórico e leva o saldo de volta ao ponto anterior do ponto de vista do negócio. Em alguns casos, o resultado nem é simétrico ao estado original (linhas de auditoria, e-mails enviados), o que reforça a diferença entre compensação lógica e rollback técnico.

Gabarito: Errado

7. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Em uma SAGA implementada com mensageria assíncrona, é comum que uma mesma mensagem seja entregue mais de uma vez, em razão de falhas de rede ou do modelo at-least-once adotado por brokers como Kafka e RabbitMQ. Para evitar efeitos colaterais duplicados nesse cenário, a prática recomendada é:

- a) configurar o broker para garantir entrega exactly-once, dispensando a necessidade de idempotência no consumidor.
- b) recusar qualquer mensagem cuja origem não seja confirmada por um canal síncrono REST paralelo ao tópico.
- c) substituir a mensageria por chamadas síncronas REST entre os participantes da SAGA.

- d) tornar o handler de cada serviço idempotente, de modo que processar a mesma mensagem N vezes produza o mesmo efeito de processá-la uma vez.
- e) descartar mensagens repetidas com base apenas na ordem de chegada, sem registrar identificadores das já processadas.

Comentários:

Idempotência é um dos pilares práticos para fazer SAGA funcionar no mundo real. A receita usual é dar a cada mensagem ou comando um identificador único (UUID, correlation-id, número da SAGA mais número da etapa) e o consumidor verifica em uma tabela de processamento se aquele id já foi tratado. Se já foi, ignora; se não foi, processa e registra. Assim, o serviço fica resiliente a retransmissões, o que é regra no mundo de mensageria distribuída.

Gabarito: Letra D

8. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Acerca da escolha do padrão SAGA em arquiteturas de microsserviços, em contraposição ao protocolo Two-Phase Commit (2PC), assinale a alternativa correta.

- a) O 2PC oferece maior disponibilidade, pois não bloqueia recursos durante a fase de preparação.
- b) O SAGA garante consistência forte imediata em todas as etapas, equivalente à do 2PC.
- c) O 2PC dispensa coordenação entre os serviços, sendo mais escalável que o SAGA em ambientes distribuídos.
- d) O SAGA exige que todos os serviços compartilhem o mesmo gerenciador transacional, de modo a coordenar o commit final.
- e) O SAGA é preferido quando os serviços possuem bancos independentes e a disponibilidade é prioritária em relação à consistência imediata.

Comentários:

A escolha do SAGA não é um capricho arquitetural: ela responde a um problema concreto. O 2PC pressupõe um coordenador global e um protocolo de prepare/commit que bloqueia recursos durante a fase de preparação, o que reduz a disponibilidade e o desempenho à medida que o número de participantes cresce. Em microsserviços, com bancos independentes (políglotas, inclusive), o custo do 2PC fica proibitivo, especialmente quando há transações de longa duração. O SAGA aceita janelas de inconsistência intermediária e usa compensação para tratar falhas, o que privilegia disponibilidade e escalabilidade.

Gabarito: Letra E

9. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Considere uma SAGA por coreografia composta pelos serviços Pedido, Pagamento e Estoque, que se comunicam por meio de eventos em um broker. No cenário em que o serviço Pagamento publica o evento PagamentoAprovado, mas o serviço Estoque falha ao reservar a mercadoria, qual é o tratamento previsto por essa variação do padrão?

- a) O serviço Estoque publica um evento ReservaFalhou, escutado por Pagamento, que dispara o estorno, e por Pedido, que marca o pedido como cancelado, executando as compensações no caminho inverso.
- b) O broker detecta a falha automaticamente e envia uma mensagem de rollback aos serviços anteriores, revertendo o estado de cada um.
- c) O serviço Pedido executa um SELECT FOR UPDATE no banco do Pagamento para forçar o estorno transacional do lançamento já confirmado.
- d) Um orquestrador central assume a coordenação assim que detecta o erro, encerrando a coreografia em andamento e migrando o fluxo para orquestração.
- e) O evento PagamentoAprovado é apagado da fila pelo broker, e essa exclusão desfaz a etapa anterior da SAGA nos serviços já consumidos.

Comentários:

Na coreografia, a recuperação de falhas se dá pela publicação de eventos de erro que disparam compensações nos serviços anteriores, exatamente o que descreve a alternativa A. O evento ReservaFalhou é a peça que costura a cadeia compensatória: Pagamento estorna a cobrança, Pedido marca o pedido como cancelado, e a ordem das compensações é a inversa da execução original. O broker não tem semântica para reverter mensagens já entregues, e o banco de um serviço não é acessado diretamente por outro.

Gabarito: Letra A

10. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Acerca das variações do padrão SAGA, julgue o item a seguir.

A orquestração tende a ser preferível em SAGAs com poucas etapas e baixo acoplamento entre serviços, enquanto a coreografia é a escolha natural quando a lógica de fluxo é complexa, com muitas ramificações condicionais, e quando se exige observabilidade centralizada do estado da transação.

Comentários:

Atenção: a assertiva inverteu os cenários. A coreografia, pelo baixo acoplamento e ausência de coordenador central, é mais indicada para fluxos curtos e relativamente simples, em que cada serviço consegue reagir a um conjunto enxuto de eventos. À medida que o fluxo ganha ramificações, etapas condicionais e necessidade de observar o estado global, a coreografia se torna difícil de seguir, pois a lógica fica espalhada por vários serviços. Nesses casos, a orquestração se beneficia da concentração do estado em um único componente, que oferece visibilidade do que cada SAGA está fazendo e facilita timeouts, retries e compensações encadeadas.

Gabarito: Errado

GABARITO

GABARITO



1. Letra A
2. Certo
3. Letra B
4. Certo

5. Letra C
6. Errado
7. Letra D
8. Letra E

9. Letra A
10. Errado

LISTA DE QUESTÕES

HORA DE PRATICAR!



1. (PROF. VINICIUS BORGES / INÉDITA - 2026)

O padrão SAGA, no contexto de microsserviços, propõe uma forma de coordenar transações que abrangem múltiplos serviços. Sobre o padrão SAGA, é correto afirmar que:

- a) divide uma transação distribuída em uma sequência de transações locais, cada uma com uma transação compensatória associada para o caso de a sequência precisar ser desfeita.
- b) impõe bloqueio distribuído entre os serviços envolvidos, no estilo two-phase locking, para garantir o isolamento da transação global.
- c) exige que todos os serviços participantes compartilhem o mesmo banco de dados relacional, de modo a preservar as propriedades ACID.
- d) substitui as transações locais por uma única transação ACID global, coordenada por um banco intermediário.
- e) elimina a necessidade de tratamento de falhas, pois delega ao gateway HTTP o rollback automático em caso de erro.

2. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Acerca do padrão SAGA aplicado a microsserviços, julgue o item a seguir.

Por dividir a transação distribuída em transações locais coordenadas a posteriori, o padrão SAGA adota o modelo de consistência eventual, em vez da consistência forte característica do protocolo Two-Phase Commit.

3. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Em uma arquitetura de microsserviços que adota o padrão SAGA na variação por coreografia, qual característica é essencial dessa abordagem?

- a) A existência de um serviço orquestrador central, que envia comandos sequenciais aos demais participantes da transação.

- b) Cada serviço participante reage a eventos publicados por outros serviços, sem coordenação centralizada do fluxo.
- c) A presença de um Transaction Manager XA, responsável por enviar prepare, commit e rollback aos recursos transacionais.
- d) O uso de chamadas síncronas REST como mecanismo principal de comunicação entre os participantes da SAGA.
- e) A serialização das transações locais por meio de locks compartilhados em um banco distribuído único.

4. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Acerca da variação por orquestração do padrão SAGA, julgue o item a seguir.

Nessa variação, existe um componente central, o orquestrador, que envia comandos aos serviços participantes e decide a próxima etapa com base nas respostas recebidas, o que facilita rastrear o estado global da SAGA.

5. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Considere uma SAGA composta pelas transações locais T1, T2 e T3, executadas nessa ordem. Se T3 falhar, o padrão prescreve a execução de transações compensatórias para reverter as etapas anteriores. Sobre essas compensações, assinale a alternativa correta.

- a) Consistem em um rollback transacional do banco, idêntico ao comando ROLLBACK do SQL, aplicado em cada serviço participante.
- b) Executam apenas sobre T3, pois somente a etapa que falhou precisa ser revertida.
- c) São executadas na ordem inversa, primeiro a compensação de T2 e depois a de T1, e devem desfazer logicamente os efeitos já confirmados.
- d) Bloqueiam os recursos das transações T1 e T2 até que o orquestrador decida abortar definitivamente a SAGA.
- e) São opcionais, pois o broker de mensagens reverte automaticamente os eventos já publicados pelos serviços.

6. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Acerca das transações compensatórias no padrão SAGA, julgue o item a seguir.

A transação compensatória executa um rollback transacional sobre o banco do serviço, revertendo bit a bit o estado anterior à transação local que se deseja desfazer.

7. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Em uma SAGA implementada com mensageria assíncrona, é comum que uma mesma mensagem seja entregue mais de uma vez, em razão de falhas de rede ou do modelo at-least-once adotado por brokers como Kafka e RabbitMQ. Para evitar efeitos colaterais duplicados nesse cenário, a prática recomendada é:

- a) configurar o broker para garantir entrega exactly-once, dispensando a necessidade de idempotência no consumidor.
- b) recusar qualquer mensagem cuja origem não seja confirmada por um canal síncrono REST paralelo ao tópico.
- c) substituir a mensageria por chamadas síncronas REST entre os participantes da SAGA.
- d) tornar o handler de cada serviço idempotente, de modo que processar a mesma mensagem N vezes produza o mesmo efeito de processá-la uma vez.
- e) descartar mensagens repetidas com base apenas na ordem de chegada, sem registrar identificadores das já processadas.

8. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Acerca da escolha do padrão SAGA em arquiteturas de microsserviços, em contraposição ao protocolo Two-Phase Commit (2PC), assinale a alternativa correta.

- a) O 2PC oferece maior disponibilidade, pois não bloqueia recursos durante a fase de preparação.
- b) O SAGA garante consistência forte imediata em todas as etapas, equivalente à do 2PC.
- c) O 2PC dispensa coordenação entre os serviços, sendo mais escalável que o SAGA em ambientes distribuídos.
- d) O SAGA exige que todos os serviços compartilhem o mesmo gerenciador transacional, de modo a coordenar o commit final.
- e) O SAGA é preferido quando os serviços possuem bancos independentes e a disponibilidade é prioritária em relação à consistência imediata.

9. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Considere uma SAGA por coreografia composta pelos serviços Pedido, Pagamento e Estoque, que se comunicam por meio de eventos em um broker. No cenário em que o serviço Pagamento publica o evento PagamentoAprovado, mas o serviço Estoque falha ao reservar a mercadoria, qual é o tratamento previsto por essa variação do padrão?

- a) O serviço Estoque publica um evento ReservaFalhou, escutado por Pagamento, que dispara o estorno, e por Pedido, que marca o pedido como cancelado, executando as compensações no caminho inverso.
- b) O broker detecta a falha automaticamente e envia uma mensagem de rollback aos serviços anteriores, revertendo o estado de cada um.
- c) O serviço Pedido executa um SELECT FOR UPDATE no banco do Pagamento para forçar o estorno transacional do lançamento já confirmado.
- d) Um orquestrador central assume a coordenação assim que detecta o erro, encerrando a coreografia em andamento e migrando o fluxo para orquestração.
- e) O evento PagamentoAprovado é apagado da fila pelo broker, e essa exclusão desfaz a etapa anterior da SAGA nos serviços já consumidos.

10.(PROF. VINICIUS BORGES / INÉDITA - 2026)

Acerca das variações do padrão SAGA, julgue o item a seguir.

A orquestração tende a ser preferível em SAGAs com poucas etapas e baixo acoplamento entre serviços, enquanto a coreografia é a escolha natural quando a lógica de fluxo é complexa, com muitas ramificações condicionais, e quando se exige observabilidade centralizada do estado da transação.

GABARITO

GABARITO



1. Letra A
2. Certo
3. Letra B
4. Certo

5. Letra C
6. Errado
7. Letra D
8. Letra E

9. Letra A
10. Errado

MICROSSERVIÇOS E O PADRÃO CIRCUIT BREAKER

Olá, pessoal! Vamos iniciar uma aula objetiva, porém densa, sobre um dos padrões mais importantes da engenharia de microsserviços: o **Circuit Breaker**. Apesar do nome técnico, a ideia central é simples e bastante didática. Em uma rede elétrica residencial, o disjuntor evita que uma sobrecarga isolada comprometa toda a instalação. Em arquiteturas distribuídas, a função é análoga: evitar que uma falha em um serviço dependente comprometa o sistema inteiro. Nesta aula, você vai entender o problema que motivou o padrão, sua máquina de estados, os principais parâmetros de configuração, a relação com padrões adjacentes como Retry, Timeout e Bulkhead, e um pouco sobre o uso prático em projetos Java/Spring Boot com a biblioteca Resilience4j. Fique tranquilo, vamos por partes.

Falha em cascata e a necessidade do disjuntor

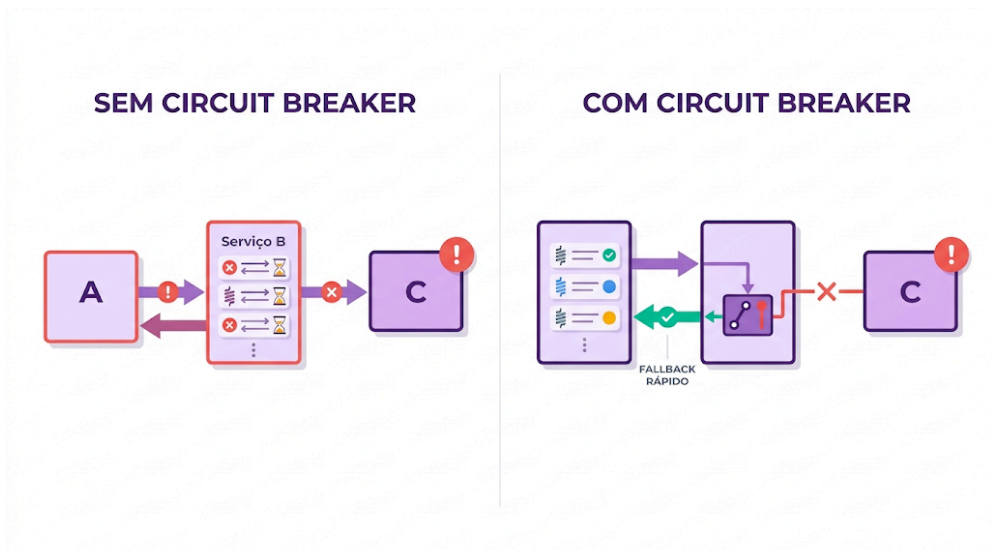
Em sistemas monolíticos, uma falha em determinada funcionalidade tende a se manter localizada no processo que a executa. Em arquiteturas de microsserviços, o cenário muda de figura, porque cada funcionalidade pode chamar uma cadeia de serviços remotos pela rede. Quando um desses serviços fica lento ou indisponível, os consumidores que dependem dele ficam aguardando respostas, mantendo threads ocupadas e conexões abertas. Com o passar dos segundos, o pool de recursos do consumidor se esgota e ele próprio passa a recusar chamadas. O problema é que, nesse ponto, o consumidor já contaminou outros serviços, que por sua vez contaminam ainda outros: instaurou-se a chamada falha em cascata.

Esse efeito é particularmente perverso porque uma instabilidade em um único componente de baixo nível pode derrubar funcionalidades de negócio que, a princípio, nem deveriam estar relacionadas. Um serviço de envio de notificações lento, por exemplo, pode bloquear threads do serviço de pedidos, que por sua vez bloqueia o gateway, que por sua vez devolve erros 5xx para o aplicativo móvel. Do ponto de vista do usuário, o sistema inteiro parou. Do ponto de vista da operação, é difícil identificar a causa raiz justamente porque a falha original fica escondida atrás de várias camadas de contaminação.

A analogia com o disjuntor elétrico

Michael Nygard, no livro *Release It!* (2007), propôs aplicar a metáfora do disjuntor elétrico ao problema das chamadas remotas. A ideia é interpor um componente entre o consumidor e o serviço dependente capaz de monitorar a saúde dessas chamadas e, ao detectar instabilidade, abrir o circuito, ou seja, interromper o tráfego durante um intervalo de espera. Enquanto o disjuntor está aberto, as chamadas falham imediatamente, sem onerar o serviço a jusante. Isso dá tempo para que o serviço dependente se recupere e impede que o consumidor consuma seus próprios recursos esperando algo que não virá.

É importante destacar que o Circuit Breaker não conserta o serviço dependente. Ele apenas isola o consumidor da indisponibilidade temporária, o que costuma ser suficiente para preservar o restante do sistema. Em conjunto com fallback, timeout e bulkhead (assunto que retomaremos adiante), o padrão compõe o conjunto mínimo de proteções recomendadas para qualquer integração síncrona entre microserviços.



Observe que, com o disjuntor em B, a falha em C deixa de se propagar para A; B passa a falhar rápido em vez de aguardar respostas que não chegariam.

Fonte: Prof. Vinicius Borges

"Falhas em sistemas distribuídos não são exceção: elas são a regra. O sistema bem-projetado é aquele que aceita a falha como dado de entrada e oferece um caminho de degradação controlada." (Nygard, M. Release It!, 2007).

Para a prova:

- Circuit Breaker é padrão de resiliência: evita que falhas se propaguem entre serviços, e não é mecanismo de balanceamento, criptografia ou transação distribuída.
- Sem disjuntor, threads e conexões do consumidor se esgotam aguardando o serviço dependente, configurando falha em cascata.
- O disjuntor não conserta o serviço a jusante: apenas isola o consumidor da indisponibilidade temporária.

HORA DE PRATICAR!



(PROF. VINICIUS BORGES / INÉDITA - 2026)

Em uma cadeia de microsserviços $A \rightarrow B \rightarrow C$, observa-se que, quando C apresenta lentidão, B passa a esgotar suas threads esperando respostas e A começa a registrar timeouts em massa. Esse efeito é tecnicamente chamado de:

- a) particionamento de rede.
- b) falha em cascata.
- c) brown-out de hardware.
- d) consistência eventual.
- e) idempotência fraca.

Comentários:

Conforme estudamos nesta seção, o esgotamento de recursos do consumidor aguardando um serviço dependente lento, com a consequente propagação da instabilidade aos consumidores acima na cadeia, é justamente o que a literatura denomina **falha em cascata**. Os demais termos não descrevem esse fenômeno: particionamento de rede é ruptura de conectividade entre nós; consistência eventual diz respeito à replicação de dados; idempotência fraca é uma propriedade de operações.

Gabarito: Letra B

A máquina de estados do disjuntor

O comportamento do Circuit Breaker é descrito por uma máquina de estados com três posições principais. Compreender o papel de cada estado e as condições de transição entre eles é o ponto de partida para configurar o padrão corretamente. Vamos ver cada um deles.

Estado fechado (closed)

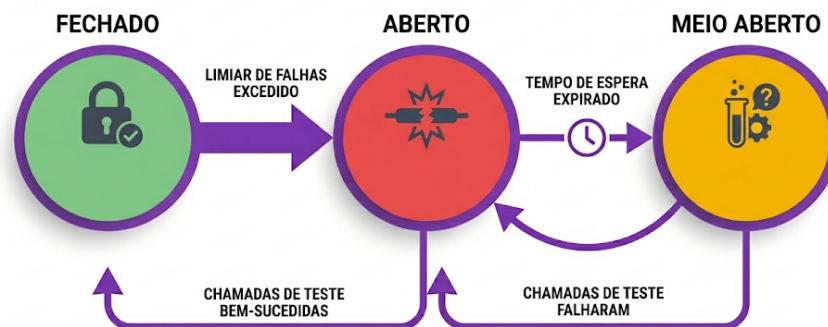
No estado fechado, o disjuntor encaminha todas as chamadas para o serviço dependente normalmente. Em paralelo, ele monitora as métricas dessas chamadas: quantas resultaram em sucesso, quantas em exceção, quantas demoraram além de um tempo limite. Esse é o estado de operação saudável; o consumidor enxerga o comportamento como se não houvesse disjuntor algum no caminho. A diferença é que, no momento em que os indicadores acumulados ultrapassam o limiar configurado, o disjuntor decide transitar para o estado aberto.

Estado aberto (open)

No estado aberto, o disjuntor bloqueia qualquer chamada nova ao serviço dependente. As tentativas falham imediatamente, geralmente com uma exceção específica do framework (em Resilience4j, a **CallNotPermittedException**). Essa fase serve a dois propósitos: o primeiro, dar tempo para o serviço a jusante se recuperar sem receber novas requisições; o segundo, liberar os recursos do consumidor, que deixa de manter threads ocupadas esperando respostas. Quando o intervalo de espera configurado (parâmetro tipicamente chamado de wait duration in open state) é atingido, o disjuntor transita para o estado semiaberto.

Estado semiaberto (half-open)

O estado semiaberto é o momento de teste. O disjuntor libera um número limitado de chamadas, configurável pelo parâmetro permitted number of calls in half-open state, para o serviço dependente. Se a maioria dessas chamadas é bem-sucedida, respeitando o failure rate threshold, o disjuntor entende que o serviço se recuperou e volta ao estado fechado. Caso contrário, ele retorna ao estado aberto e reinicia o intervalo de espera. Esse mecanismo evita reabrir o tráfego integralmente para um serviço que ainda pode estar instável, e por isso é fundamental para a estabilidade da operação.



Repare na transição em duas etapas para reabrir o tráfego: o disjuntor não pula direto de aberto para fechado; ele passa pelo semiaberto, em que valida a recuperação com poucas chamadas de prova.

Fonte: Prof. Vinicius Borges

Estado	Chamadas ao serviço dependente	Critério de transição
Closed	Encaminhadas normalmente	Limiar de falhas ou de chamadas lentas excedido -> Open
Open	Falham imediatamente (fail-fast)	Wait duration expira -> Half-Open
Half-Open	Número limitado de chamadas de teste	Sucesso -> Closed; falha -> Open

Para a prova:

- Os três estados são Closed, Open e Half-Open; cada um com regra própria para encaminhar ou bloquear chamadas.
- A transição Open -> Half-Open ocorre por tempo (wait duration in open state), não por novas chamadas.
- A transição Half-Open -> Closed ou -> Open depende do resultado de poucas chamadas de teste, não do tráfego inteiro.

HORA DE PRATICAR!



(PROF. VINICIUS BORGES / INÉDITA - 2026)

Um Circuit Breaker no estado aberto recebe uma nova requisição de um consumidor. Sobre o comportamento esperado nesse cenário, é correto afirmar:

- a) o disjuntor encaminha a chamada e registra o resultado para definir o próximo estado.
- b) o disjuntor falha imediatamente a chamada, geralmente com uma exceção do tipo `CallNotPermittedException`, sem onerar o serviço dependente.
- c) o disjuntor enfileira a chamada até o término do `wait duration in open state`.
- d) o disjuntor transita imediatamente para o estado fechado e libera o tráfego.
- e) o disjuntor delega a decisão ao serviço de configuração centralizado.

Comentários:

Realmente, como vimos nesta seção, no estado **aberto** o disjuntor rejeita chamadas imediatamente, configurando o comportamento chamado `fail-fast`. Em `Resilience4j`, essa rejeição se manifesta como `CallNotPermittedException`.

Gabarito: Letra B

Configuração: janelas, limiares e métricas

A escolha dos parâmetros do disjuntor é o que diferencia uma configuração eficaz de uma configuração que ou abre cedo demais (interrompendo o tráfego de forma desnecessária) ou tarde demais (deixando a falha se propagar). Os principais parâmetros que você precisa conhecer são a janela de coleta, o failure rate threshold, o slow call rate threshold, o número mínimo de chamadas para a decisão, o wait duration in open state e o número de chamadas permitidas em half-open. Vamos analisá-los.

Janela deslizante e número mínimo de chamadas

O disjuntor não decide com base em uma única chamada. Ele acumula resultados em uma janela deslizante, configurável em duas modalidades: por contagem (COUNT_BASED), em que se considera as últimas N chamadas, ou por tempo (TIME_BASED), em que se considera as chamadas dos últimos N segundos. Existe também o parâmetro minimum number of calls, que define um piso de chamadas antes de qualquer decisão de transição. Sem ele, uma única falha em uma janela quase vazia poderia abrir o disjuntor, o que claramente não é o comportamento desejado.

Failure rate threshold e slow call rate threshold

O failure rate threshold define a porcentagem máxima admitida de chamadas com falha dentro da janela. Quando essa porcentagem é atingida ou ultrapassada, o disjuntor abre. Mas existe uma armadilha: um serviço dependente pode estar tecnicamente respondendo, sem lançar exceções, e ainda assim levar dezenas de segundos por chamada. Para cobrir esse caso, o Resilience4j oferece o slow call rate threshold: chamadas cuja duração ultrapassa o slow call duration threshold são contadas como lentas, e a porcentagem dessas chamadas alimenta um segundo critério de abertura. Os dois critérios funcionam em paralelo: se qualquer deles estoura, o disjuntor abre.



Observe que o disjuntor não conta apenas as falhas absolutas; o que importa é a porcentagem dentro da janela, comparada ao failure rate threshold configurado.

Fonte: Prof. Vinicius Borges

Wait duration e chamadas em half-open

O parâmetro `wait duration in open state` define quanto tempo o disjuntor permanece no estado aberto antes de transitar para semiaberto. Valores típicos vão de alguns segundos a alguns minutos, e dependem do tempo médio que o serviço dependente leva para se recuperar de uma falha. O parâmetro `permitted number of calls in half-open state` define quantas chamadas de teste são liberadas naquela fase. Um valor baixo, como 3 ou 5, é o suficiente: o objetivo é apenas medir se o serviço voltou a funcionar, não restabelecer plenamente o tráfego, que só ocorre após a transição para fechado.

Parâmetro	Função	Valor inicial razoável
<code>slidingWindowType</code>	Define se a janela é por contagem ou tempo	COUNT_BASED
<code>slidingWindowSize</code>	Tamanho da janela (chamadas ou segundos)	100 (count) ou 60s (time)
<code>minimumNumberOfCalls</code>	Piso de chamadas antes de decidir	10 a 20
<code>failureRateThreshold</code>	Porcentagem de falhas para abrir	50%
<code>slowCallRateThreshold</code>	Porcentagem de chamadas lentas para abrir	100% (desativa) ou 80%
<code>slowCallDurationThreshold</code>	Tempo a partir do qual a chamada é lenta	2s a 5s
<code>waitDurationInOpenState</code>	Tempo de espera antes do half-open	30s a 60s
<code>permittedNumberOfCallsInHalfOpenState</code>	Chamadas de teste no half-open	3 a 10

Para a prova:

- A janela pode ser COUNT_BASED (últimas N chamadas) ou TIME_BASED (últimos N segundos).
- O `minimumNumberOfCalls` evita decisões precipitadas em janelas quase vazias.
- Failure rate e slow call rate são critérios complementares: qualquer um deles, isoladamente, pode disparar a abertura.
- Wait duration in open state controla quando o disjuntor entra em half-open; `permittedNumberOfCallsInHalfOpenState` controla quantas chamadas de teste são feitas naquela fase.

HORA DE PRATICAR!



(PROF. VINICIUS BORGES / INÉDITA - 2026)

Considere um Circuit Breaker com janela COUNT_BASED de tamanho 20, minimumNumberOfCalls igual a 10, failureRateThreshold de 50% e slowCallRateThreshold desativado. Em uma janela com 8 chamadas, sendo 4 falhas, o disjuntor:

- a) abre, porque 50% das chamadas falharam.
- b) permanece fechado, porque o piso de minimumNumberOfCalls (10) ainda não foi atingido.
- c) transita diretamente para half-open, ignorando os limiares.
- d) abre, porque a janela atingiu 50% antes do tamanho total.
- e) lança CallNotPermittedException, mesmo no estado fechado.

Comentários:

Realmente, como vimos, o parâmetro **minimumNumberOfCalls** impede que o disjuntor decida sobre transição antes que haja chamadas suficientes na janela. Com apenas 8 chamadas registradas, o disjuntor não avalia o failureRateThreshold, ainda que aritmeticamente já estejamos em 50%. Esse parâmetro existe justamente para evitar decisões precipitadas com base em amostragem insuficiente.

Gabarito: Letra B

Padrões adjacentes: Retry, Timeout, Bulkhead e Fallback

O Circuit Breaker raramente é usado sozinho. Ele integra um conjunto de padrões de resiliência que se complementam, e confundir um com outro é uma armadilha comum em prova. Vamos olhar cada um deles e como se relacionam com o disjuntor.

Retry: insistir em falhas transitórias

O padrão Retry repete uma chamada falha um número configurado de vezes, geralmente com backoff exponencial entre as tentativas. Ele parte do pressuposto de que a falha é transitória, como um pico de latência ou uma desconexão momentânea, e que insistir um pouco pode ser suficiente para obter sucesso. Em ambientes distribuídos, no entanto, retry sem disjuntor é perigoso: várias instâncias do consumidor podem repetir chamadas em paralelo e amplificar a carga sobre um serviço já em degradação. A composição recomendada é envolver o retry pelo circuit breaker: enquanto o disjuntor estiver aberto, nenhuma nova tentativa ocorre, evitando o efeito conhecido como retry storm.

Timeout: limite explícito de espera

Toda chamada remota deve ter timeout. Sem ele, a thread do consumidor pode ficar presa indefinidamente esperando uma resposta que não vem, e o Circuit Breaker nem chega a registrar uma falha (afinal, do ponto de vista do disjuntor, a chamada nem terminou). Definir timeouts adequados é pré-requisito para que o disjuntor consiga atuar. Em Resilience4j, o módulo TimeLimiter cuida desse aspecto, e ele costuma ser combinado com o CircuitBreaker no mesmo decorator.

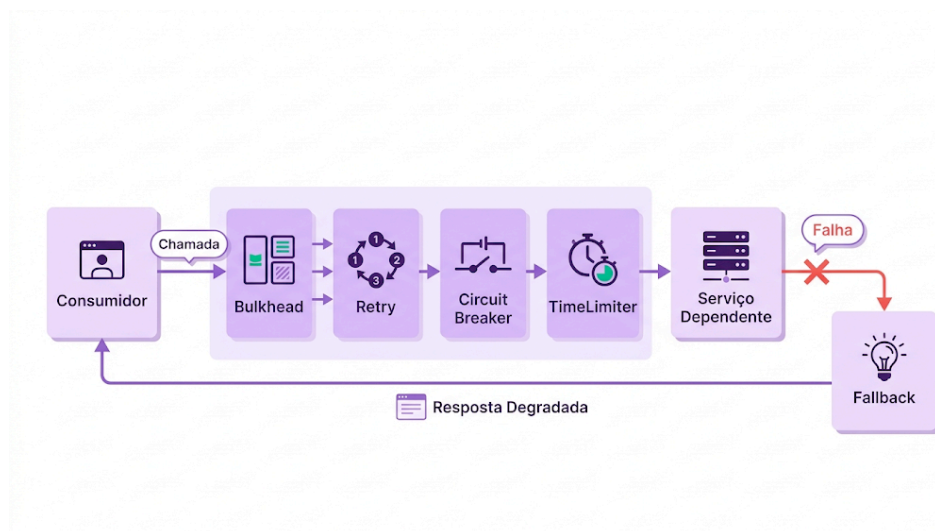
Bulkhead: isolamento de recursos

O padrão Bulkhead, inspirado nos compartimentos estanques de navios, limita o número de chamadas concorrentes a um determinado recurso. Em um cenário típico, atribui-se um pool de threads exclusivo para cada integração; se uma dessas integrações entra em degradação, suas threads dedicadas se esgotam, mas as demais integrações permanecem operando normalmente. É uma camada de proteção ortogonal ao Circuit Breaker: enquanto o disjuntor protege o serviço dependente (rejeitando chamadas durante a recuperação), o bulkhead protege o próprio consumidor (impedindo que uma integração monopolize os recursos). Os dois são complementares.

Fallback: caminho alternativo de execução

Quando o disjuntor está aberto ou quando uma chamada falha, o consumidor precisa decidir o que devolver para quem o chamou. O fallback é justamente o caminho alternativo: pode retornar um valor padrão razoável, dados de um cache local, uma resposta degradada com aviso ao usuário, ou ainda redirecionar para um serviço de contingência. Em termos práticos, é o fallback que

transforma uma indisponibilidade dura (erro 500 para o usuário) em uma indisponibilidade controlada (lista vazia, mensagem amigável). Em Resilience4j, ele é declarado por anotação @Fallback ou na composição funcional do decorator.



Observe a ordem de composição em Resilience4j: Bulkhead na borda externa, depois Retry, depois CircuitBreaker, depois TimeLimiter; o Fallback fica como rede de segurança final.

Fonte: Prof. Vinicius Borges

Padrão	O que faz	Quando combinar com Circuit Breaker
Retry	Repete tentativas em caso de falha transitória	Sempre dentro do disjuntor, para evitar retry storm
Timeout	Limita o tempo máximo de espera de uma chamada	Pré-requisito; sem ele, o disjuntor não consegue atuar
Bulkhead	Isola pools de recursos por integração	Ortogonal; protege o consumidor enquanto o disjuntor protege o serviço dependente
Fallback	Caminho alternativo de execução em caso de falha	Acionado quando o disjuntor está aberto ou a chamada falha

Para a prova:

- Retry não é sinônimo de Circuit Breaker: retry insiste em falhas transitórias, disjuntor bloqueia chamadas durante a recuperação.
- Bulkhead também não é sinônimo de Circuit Breaker: bulkhead limita concorrência, disjuntor monitora taxa de falhas e latência.
- Timeout é pré-requisito do Circuit Breaker; chamadas sem limite de espera não permitem ao disjuntor avaliar a saúde do serviço dependente.

- Fallback complementa o disjuntor entregando resposta degradada quando ele está aberto.

HORA DE PRATICAR!



(PROF. VINICIUS BORGES / INÉDITA - 2026)

Acerca da relação entre os padrões Retry e Circuit Breaker em uma arquitetura de microsserviços, julgue o item: como ambos têm o mesmo objetivo de tratar falhas remotas, a aplicação simultânea dos dois é redundante e deve ser evitada.

Comentários:

Nada disso! Conforme estudamos nesta seção, **Retry** e **Circuit Breaker** não têm o mesmo objetivo. Retry insiste em falhas transitórias; Circuit Breaker bloqueia chamadas durante a recuperação do serviço dependente. A composição recomendada combina os dois, com o retry envolvido pelo disjuntor: enquanto o disjuntor estiver aberto, nenhum retry é executado, o que evita o efeito de retry storm. Considerar a aplicação simultânea redundante é tecnicamente equivocado.

Gabarito: Errado

QUESTÕES COMENTADAS

HORA DE PRATICAR!



1. (PROF. VINICIUS BORGES / INÉDITA - 2026)

O padrão Circuit Breaker, amplamente utilizado em arquiteturas de microsserviços, tem como propósito principal:

- a) impedir que falhas localizadas em um serviço dependente se propaguem para o consumidor, interrompendo temporariamente as chamadas e permitindo que o sistema se recupere.
- b) substituir o uso de balanceadores de carga em ambientes distribuídos, distribuindo requisições entre instâncias saudáveis.
- c) garantir transações distribuídas atômicas entre microsserviços, atuando como um coordenador 2PC.
- d) criptografar o tráfego entre serviços, substituindo o uso de TLS no service mesh.
- e) realizar deploy progressivo de novas versões, redirecionando parte do tráfego para a versão nova.

Comentários:

O Circuit Breaker, popularizado por Michael Nygard no livro Release It!, atua como um disjuntor elétrico aplicado à comunicação entre serviços. Quando o serviço dependente começa a falhar acima de um limiar configurado, o disjuntor abre, e as chamadas subsequentes falham imediatamente, sem onerar o serviço problemático. Isso evita o efeito conhecido como falha em cascata, em que uma instabilidade local consome threads e conexões de todos os consumidores. As demais alternativas confundem o padrão com responsabilidades de outros componentes: balanceamento de carga, transações distribuídas, criptografia ou deploy progressivo.

Gabarito: Letra A

2. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Acerca do padrão Circuit Breaker, julgue o item a seguir: o disjuntor opera em três estados principais, denominados fechado (closed), aberto (open) e semiaberto (half-open), cada um com regras próprias para encaminhar ou bloquear as chamadas ao serviço dependente.

Comentários:

Isso mesmo! A máquina de estados do Circuit Breaker é formada justamente pelos três estados citados. No estado fechado, as chamadas seguem normalmente para o serviço dependente, mas o disjuntor monitora falhas e latência. Quando o limiar configurado é ultrapassado, o disjuntor transita para o estado aberto e passa a falhar rápido, sem encaminhar requisições. Após um intervalo de espera, o disjuntor entra em semiaberto e libera um número reduzido de chamadas de teste; se essas chamadas forem bem-sucedidas, o disjuntor volta ao estado fechado.

Gabarito: Certo

3. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Sobre o estado semiaberto (half-open) do Circuit Breaker em uma implementação como o Resilience4j, é correto afirmar que:

- a) o disjuntor bloqueia todas as chamadas e responde imediatamente com a função de fallback, sem encaminhar requisições reais.
- b) o disjuntor permite um número limitado de chamadas de teste para o serviço dependente e decide, com base no resultado dessas chamadas, se retorna ao estado fechado ou volta para o estado aberto.
- c) o disjuntor encaminha o tráfego normalmente, mas registra um log adicional para fins de auditoria.
- d) o disjuntor delega a decisão de roteamento a um serviço externo de orquestração, geralmente um service mesh.
- e) o disjuntor descarta o histórico de falhas anteriores e reinicia o contador, comportando-se como se tivesse acabado de ser criado.

Comentários:

Realmente, o estado semiaberto funciona como uma fase de teste. Após o tempo de espera configurado (wait duration in open state), o disjuntor sai do estado aberto e entra em half-open, permitindo um número configurável de chamadas de prova (permitted number of calls in half-open state). Se a taxa de sucesso dessas chamadas atinge o limiar esperado, o disjuntor entende que o serviço dependente se recuperou e volta para o estado fechado. Caso contrário, retorna ao estado aberto e reinicia o tempo de espera. Esse mecanismo evita reabrir o tráfego de uma vez só para um serviço que pode ainda estar instável.

Gabarito: Letra B

4. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Acerca dos padrões de resiliência em microserviços, julgue o item: o Circuit Breaker é equivalente ao padrão Retry, diferenciando-se apenas pelo nome, visto que ambos executam novas tentativas automáticas de chamada quando ocorre uma falha no serviço dependente.

Comentários:

Nada disso! Embora os dois padrões cuidem de falhas em chamadas remotas, eles têm objetivos opostos. O Retry repete a mesma chamada algumas vezes, partindo do pressuposto de que a falha é transitória, como um pico momentâneo de latência ou uma desconexão pontual. O Circuit Breaker, ao contrário, parte do pressuposto de que o serviço dependente está em degradação e que insistir em chamá-lo só piora a situação; por isso, ele bloqueia as chamadas durante um intervalo configurado. Na prática, retry e circuit breaker costumam ser combinados, com o retry envolvido pelo disjuntor, de modo que tentativas repetidas não derrubem o serviço.

Gabarito: Errado

5. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Considere um Circuit Breaker configurado com janela deslizante baseada em contagem (sliding window of type COUNT_BASED) com tamanho 100 e failure rate threshold de 50%. Suponha que, dentro dessa janela, ocorram 60 chamadas com sucesso e 40 chamadas com falha. Sobre o comportamento do disjuntor nesse cenário, assinale a alternativa correta:

- a) o disjuntor transita imediatamente para o estado aberto, porque qualquer falha registrada já dispara a abertura.
- b) o disjuntor transita para o estado semiaberto, pois failure rate threshold se aplica apenas ao half-open.
- c) o disjuntor permanece no estado fechado, porque a taxa de falhas de 40% está abaixo do limiar configurado de 50%.
- d) o disjuntor transita para o estado aberto, porque a taxa absoluta de falhas é maior que 30%.
- e) o disjuntor lança uma exceção do tipo CallNotPermittedException, pois 40 falhas em sequência ultrapassam o tamanho da janela.

Comentários:

O failure rate threshold só dispara a transição para o estado aberto quando a porcentagem de falhas dentro da janela atinge ou ultrapassa o valor configurado. No cenário descrito, 40 falhas em 100 chamadas correspondem a uma taxa de 40%, abaixo do limiar de 50%. Por isso, o disjuntor permanece fechado, encaminhando o tráfego normalmente.

6. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Julgue o item a seguir sobre o uso de fallback associado ao Circuit Breaker: o fallback é um caminho alternativo de execução, acionado quando o disjuntor está aberto ou quando a chamada falha, podendo retornar um valor padrão, dados em cache ou redirecionar para um serviço alternativo, com o objetivo de preservar a experiência do usuário mesmo diante de indisponibilidade do serviço dependente.

Comentários:

Realmente, o fallback complementa o Circuit Breaker oferecendo uma resposta degradada quando o disjuntor está aberto. Em vez de propagar uma exceção até o consumidor final, a aplicação executa uma rotina alternativa que pode retornar um valor default razoável (por exemplo, uma lista vazia de recomendações), servir dados de um cache local com TTL relaxado ou consultar um serviço de contingência. Esse padrão evita que o usuário receba uma página de erro em situações em que uma experiência degradada ainda é útil. Em Resilience4j, o fallback é declarado por anotação ou na composição funcional do decorator.

Gabarito: Certo

7. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Sobre as métricas que podem disparar a abertura de um Circuit Breaker no Resilience4j, assinale a alternativa correta:

- a) somente a quantidade absoluta de exceções lançadas no último minuto influencia a abertura.
- b) o disjuntor considera apenas códigos de status HTTP 5xx, ignorando latência.
- c) a abertura é controlada exclusivamente pelo número de chamadas em paralelo, em padrão similar ao bulkhead.
- d) além da failure rate, o disjuntor também observa a slow call rate, que mede a porcentagem de chamadas cuja duração ultrapassou um limite configurado (slow call duration threshold).
- e) a abertura depende somente da taxa de timeouts; outros tipos de falha são ignorados pelo disjuntor.

Comentários:

Como vimos, o Resilience4j permite que o disjuntor abra não apenas com base na proporção de chamadas que falham, mas também com base na proporção de chamadas consideradas lentas. A

slow call rate threshold define a porcentagem de chamadas que excedem o slow call duration threshold; quando essa porcentagem é alcançada, o disjuntor abre mesmo que poucas chamadas tenham resultado em exceção. Esse critério é importante porque um serviço dependente pode estar tecnicamente respondendo, mas com latência tão alta que prejudica os consumidores.

Gabarito: Letra D

8. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Julgue o item a seguir: o padrão Bulkhead, em sistemas distribuídos, é um sinônimo do padrão Circuit Breaker, sendo ambos responsáveis por monitorar a taxa de falhas de um serviço dependente e bloquear chamadas durante um intervalo de espera quando essa taxa ultrapassa um limiar configurado.

Comentários:

Cuidado, pessoal! Bulkhead e Circuit Breaker são padrões distintos e complementares. O Bulkhead, inspirado nos compartimentos estanques de navios, limita o número de chamadas concorrentes a um recurso, isolando o consumo de threads ou conexões; se o limite é atingido, novas chamadas são rejeitadas ou enfileiradas, sem afetar os demais consumidores do sistema. Já o Circuit Breaker monitora taxa de falhas e latência para decidir abrir ou fechar o disjuntor. Os dois podem ser usados juntos: o bulkhead protege o consumidor contra esgotamento de recursos, e o circuit breaker protege o serviço dependente contra novas chamadas durante a recuperação. Confundi-los é uma pegadinha frequente.

Gabarito: Errado

9. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Em projetos Java/Spring Boot atuais, a biblioteca recomendada para implementar o padrão Circuit Breaker, em substituição ao Netflix Hystrix (em modo de manutenção desde 2018), é o Resilience4j. Sobre essa biblioteca, assinale a alternativa correta:

- a) o Resilience4j depende obrigatoriamente do RxJava como runtime reativo, não funcionando em aplicações tradicionais baseadas em Servlet.
- b) o Resilience4j implementa apenas o padrão Circuit Breaker, sem oferecer Retry, Rate Limiter, Bulkhead ou Time Limiter.
- c) o Resilience4j substitui o Spring Cloud Gateway, atuando como API gateway entre o cliente e os microsserviços.

d) o Resilience4j armazena o estado do disjuntor exclusivamente em banco de dados relacional compartilhado entre todas as instâncias do serviço.

e) o Resilience4j expõe eventos do disjuntor (como `CircuitBreakerOnStateTransitionEvent`) que podem ser consumidos para integração com sistemas de observabilidade, permitindo o monitoramento das transições de estado.

Comentários:

Realmente, o Resilience4j adota uma arquitetura baseada em eventos. Cada disjuntor publica eventos como `CircuitBreakerOnStateTransitionEvent`, `OnErrorEvent`, `OnSuccessEvent` e `OnSlowCallRateExceededEvent`, que podem ser consumidos por listeners da aplicação ou exportados para Micrometer, Prometheus e Grafana. Esse recurso é central para observabilidade do padrão em produção. As demais alternativas trazem afirmações incorretas: o Resilience4j funciona em aplicações tradicionais sem RxJava, oferece vários módulos além do Circuit Breaker, não substitui o API gateway e mantém o estado em memória da própria instância (ou em estruturas compartilhadas como Redis, mediante extensão).

Gabarito: Letra E

10. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Considere uma arquitetura de microsserviços em que o serviço A depende do serviço B, que por sua vez depende do serviço C. Caso o serviço C entre em degradação e o serviço B não tenha um Circuit Breaker configurado para suas chamadas a C, é possível que threads do serviço B fiquem bloqueadas aguardando respostas, levando o serviço A a observar timeouts e, por extensão, a degradação do sistema como um todo, em um efeito conhecido como falha em cascata. Julgue o item: a aplicação do Circuit Breaker em B, em conjunto com timeouts adequados e isolamento por bulkhead, é uma estratégia recomendada para mitigar esse risco.

Comentários:

Isso mesmo! O cenário descrito é o exemplo clássico de propagação de falhas em cadeia. Sem disjuntor, o serviço B mantém threads ocupadas esperando o serviço C responder; conforme novas requisições chegam, o pool de threads se esgota e B passa a recusar chamadas, contaminando o serviço A. A combinação recomendada inclui: definir timeout explícito para a chamada de B a C, de modo que a thread não fique presa indefinidamente; configurar um Circuit Breaker em B que abra quando a taxa de falhas ou de chamadas lentas a C ultrapassar um limiar; e aplicar Bulkhead para isolar o pool de threads usado nessa integração, evitando que o consumo descontrolado afete outras funcionalidades de B. Essa composição é tratada na literatura de resiliência como conjunto mínimo para ambientes distribuídos.

Gabarito: Certo

GABARITO

GABARITO



1. Letra A
2. Certo
3. Letra B
4. Errado

5. Letra C
6. Certo
7. Letra D
8. Errado

9. Letra E
10. Certo

LISTA DE QUESTÕES

HORA DE PRATICAR!



1. (PROF. VINICIUS BORGES / INÉDITA - 2026)

O padrão Circuit Breaker, amplamente utilizado em arquiteturas de microsserviços, tem como propósito principal:

- a) impedir que falhas localizadas em um serviço dependente se propaguem para o consumidor, interrompendo temporariamente as chamadas e permitindo que o sistema se recupere.
- b) substituir o uso de balanceadores de carga em ambientes distribuídos, distribuindo requisições entre instâncias saudáveis.
- c) garantir transações distribuídas atômicas entre microsserviços, atuando como um coordenador 2PC.
- d) criptografar o tráfego entre serviços, substituindo o uso de TLS no service mesh.
- e) realizar deploy progressivo de novas versões, redirecionando parte do tráfego para a versão nova.

2. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Acerca do padrão Circuit Breaker, julgue o item a seguir: o disjuntor opera em três estados principais, denominados fechado (closed), aberto (open) e semiaberto (half-open), cada um com regras próprias para encaminhar ou bloquear as chamadas ao serviço dependente.

3. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Sobre o estado semiaberto (half-open) do Circuit Breaker em uma implementação como o Resilience4j, é correto afirmar que:

- a) o disjuntor bloqueia todas as chamadas e responde imediatamente com a função de fallback, sem encaminhar requisições reais.
- b) o disjuntor permite um número limitado de chamadas de teste para o serviço dependente e decide, com base no resultado dessas chamadas, se retorna ao estado fechado ou volta para o estado aberto.

- c) o disjuntor encaminha o tráfego normalmente, mas registra um log adicional para fins de auditoria.
- d) o disjuntor delega a decisão de roteamento a um serviço externo de orquestração, geralmente um service mesh.
- e) o disjuntor descarta o histórico de falhas anteriores e reinicia o contador, comportando-se como se tivesse acabado de ser criado.

4. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Acerca dos padrões de resiliência em microsserviços, julgue o item: o Circuit Breaker é equivalente ao padrão Retry, diferenciando-se apenas pelo nome, visto que ambos executam novas tentativas automáticas de chamada quando ocorre uma falha no serviço dependente.

5. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Considere um Circuit Breaker configurado com janela deslizante baseada em contagem (sliding window of type COUNT_BASED) com tamanho 100 e failure rate threshold de 50%. Suponha que, dentro dessa janela, ocorram 60 chamadas com sucesso e 40 chamadas com falha. Sobre o comportamento do disjuntor nesse cenário, assinale a alternativa correta:

- a) o disjuntor transita imediatamente para o estado aberto, porque qualquer falha registrada já dispara a abertura.
- b) o disjuntor transita para o estado semiaberto, pois failure rate threshold se aplica apenas ao half-open.
- c) o disjuntor permanece no estado fechado, porque a taxa de falhas de 40% está abaixo do limiar configurado de 50%.
- d) o disjuntor transita para o estado aberto, porque a taxa absoluta de falhas é maior que 30%.
- e) o disjuntor lança uma exceção do tipo CallNotPermittedException, pois 40 falhas em sequência ultrapassam o tamanho da janela.

6. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Julgue o item a seguir sobre o uso de fallback associado ao Circuit Breaker: o fallback é um caminho alternativo de execução, acionado quando o disjuntor está aberto ou quando a chamada falha, podendo retornar um valor padrão, dados em cache ou redirecionar para um serviço alternativo, com o objetivo de preservar a experiência do usuário mesmo diante de indisponibilidade do serviço dependente.

7. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Sobre as métricas que podem disparar a abertura de um Circuit Breaker no Resilience4j, assinale a alternativa correta:

- a) somente a quantidade absoluta de exceções lançadas no último minuto influencia a abertura.
- b) o disjuntor considera apenas códigos de status HTTP 5xx, ignorando latência.
- c) a abertura é controlada exclusivamente pelo número de chamadas em paralelo, em padrão similar ao bulkhead.
- d) além da failure rate, o disjuntor também observa a slow call rate, que mede a porcentagem de chamadas cuja duração ultrapassou um limite configurado (slow call duration threshold).
- e) a abertura depende somente da taxa de timeouts; outros tipos de falha são ignorados pelo disjuntor.

8. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Julgue o item a seguir: o padrão Bulkhead, em sistemas distribuídos, é um sinônimo do padrão Circuit Breaker, sendo ambos responsáveis por monitorar a taxa de falhas de um serviço dependente e bloquear chamadas durante um intervalo de espera quando essa taxa ultrapassa um limiar configurado.

9. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Em projetos Java/Spring Boot atuais, a biblioteca recomendada para implementar o padrão Circuit Breaker, em substituição ao Netflix Hystrix (em modo de manutenção desde 2018), é o Resilience4j. Sobre essa biblioteca, assinale a alternativa correta:

- a) o Resilience4j depende obrigatoriamente do RxJava como runtime reativo, não funcionando em aplicações tradicionais baseadas em Servlet.
- b) o Resilience4j implementa apenas o padrão Circuit Breaker, sem oferecer Retry, Rate Limiter, Bulkhead ou Time Limiter.
- c) o Resilience4j substitui o Spring Cloud Gateway, atuando como API gateway entre o cliente e os microsserviços.
- d) o Resilience4j armazena o estado do disjuntor exclusivamente em banco de dados relacional compartilhado entre todas as instâncias do serviço.

e) o Resilience4j expõe eventos do disjuntor (como `CircuitBreakerOnStateTransitionEvent`) que podem ser consumidos para integração com sistemas de observabilidade, permitindo o monitoramento das transições de estado.

10. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Considere uma arquitetura de microsserviços em que o serviço A depende do serviço B, que por sua vez depende do serviço C. Caso o serviço C entre em degradação e o serviço B não tenha um Circuit Breaker configurado para suas chamadas a C, é possível que threads do serviço B fiquem bloqueadas aguardando respostas, levando o serviço A a observar timeouts e, por extensão, a degradação do sistema como um todo, em um efeito conhecido como falha em cascata. Julgue o item: a aplicação do Circuit Breaker em B, em conjunto com timeouts adequados e isolamento por bulkhead, é uma estratégia recomendada para mitigar esse risco.

GABARITO

GABARITO



1. Letra A
2. Certo
3. Letra B
4. Errado

5. Letra C
6. Certo
7. Letra D
8. Errado

9. Letra E
10. Certo

MICROSSERVIÇOS E BACKEND FOR FRONTEND (BFF)

Olá, pessoal! Vamos iniciar nosso estudo sobre um dos temas mais presentes nos editais modernos de Tecnologia da Informação: **microsserviços** e o padrão **Backend For Frontend (BFF)**. O assunto vai aparecer em contextos diversos: arquitetura de sistemas, governança de TI, qualidade de software e desenvolvimento web. Em todos eles, a banca cobra a mesma essência: o que muda quando passamos do monolito para uma malha de serviços pequenos, e como adaptar essa malha às necessidades de cada cliente.

Para você acompanhar com tranquilidade, organizei o material em quatro blocos. Primeiro vamos entender o que define um microsserviço e por que ele resolve (ou cria) certos problemas. Depois discutiremos como esses serviços conversam entre si, com ênfase em comunicação síncrona, assíncrona e no padrão Saga. Em seguida, entramos no coração do nosso tema: API Gateway, Backend For Frontend e a relação entre eles. Encerramos com anti-patterns, Service Mesh e dicas práticas.

Arquitetura de Microsserviços

Conceito e características

A arquitetura de **microsserviços** é um estilo arquitetural em que uma aplicação é estruturada como um conjunto de pequenos serviços, cada um executando em seu próprio processo, comunicando-se por meio de mecanismos leves (tipicamente APIs HTTP/REST ou gRPC) e podendo ser implantado de forma independente. Essa definição, popularizada por Martin Fowler e James Lewis em 2014, contrasta com o estilo monolítico, em que toda a funcionalidade reside em um único artefato implantável.

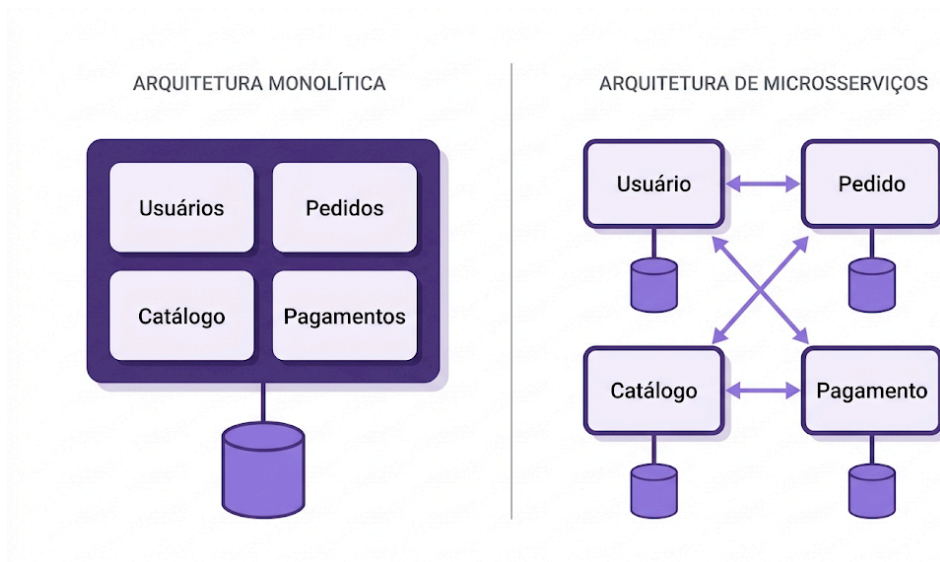
Algumas características aparecem com regularidade nas provas. Cada microsserviço tem responsabilidade bem delimitada (alinhada a um **contexto delimitado** do Domain-Driven Design); é dono dos próprios dados, não compartilhando o banco com outros serviços; pode ser desenvolvido em uma stack tecnológica diferente da escolhida pelos vizinhos (heterogeneidade tecnológica); é versionado e implantado por seu próprio time, sem coordenação obrigatória com outros times.

"Microsserviços são uma abordagem para desenvolver uma única aplicação como uma suíte de pequenos serviços, cada um rodando em seu próprio processo e se comunicando com mecanismos leves." (Martin Fowler, 2014)

Monolito versus microsserviços

Antes de seguir, vale comparar lado a lado os dois estilos arquiteturais, porque a banca costuma cobrar essa contraposição com perguntas pontuais.

Critério	Monolito	Microsserviços
Unidade de deploy	Um único artefato para toda a aplicação	Um artefato por serviço, com deploy independente
Stack tecnológica	Tipicamente uma única stack (linguagem, framework, banco)	Cada serviço escolhe sua stack (heterogeneidade)
Escalabilidade	Replica-se a aplicação inteira	Escala-se apenas o serviço sob carga
Acoplamento entre times	Alto: times compartilham a base de código e o ciclo de release	Baixo: cada time é dono do seu serviço, com contratos versionados
Latência interna	Chamadas in-process (rápidas)	Chamadas de rede (latência maior, falhas possíveis)
Complexidade operacional	Menor: poucos artefatos para implantar e monitorar	Maior: orquestração, observabilidade distribuída, mais pontos de falha



Repare que no monolito os módulos se comunicam in-process (linhas internas), enquanto nos microserviços a comunicação atravessa a rede; é essa fronteira de rede que muda toda a discussão de latência, falhas e governança.

Fonte: Prof. Vinicius Borges

Note que microserviços não substituem o monolito em todos os cenários. Para aplicações pequenas, com um único time e domínio simples, um **monolito modular** bem desenhado costuma ser mais produtivo. A migração para microserviços faz sentido quando há complexidade real: múltiplos times disputando o mesmo deploy, partes da aplicação com cargas muito diferentes, ou necessidade de evoluir tecnologias por subdomínio.

Para a prova:

- Microserviços = serviços pequenos, autônomos, com deploy independente, comunicação leve e dados próprios.
- Não é regra de tamanho de linhas de código; o critério é responsabilidade bem delimitada (DDD).
- Heterogeneidade tecnológica é vantagem do estilo, não obrigação.
- Pagam-se custos operacionais maiores em troca de autonomia de times e escalabilidade granular.

HORA DE PRATICAR!



(PROF. VINICIUS BORGES / INÉDITA - 2026)

Acerca de arquitetura de microsserviços, julgue o item a seguir: microsserviços são um estilo arquitetural em que uma aplicação é estruturada como um conjunto de pequenos serviços autônomos, cada um executando em seu próprio processo e comunicando-se por meio de mecanismos leves, frequentemente APIs HTTP.

Comentários:

Conforme estudamos nesta seção, o item reproduz a definição clássica de Fowler e Lewis. Os elementos centrais estão todos lá: serviços pequenos, autonomia (processo próprio e dados próprios), comunicação leve (HTTP/REST ou gRPC) e independência de implantação.

Gabarito: Certo

Comunicação entre Microserviços

Uma vez que decompomos a aplicação em vários serviços, surge a pergunta inevitável: como eles conversam? O tema tem duas dimensões principais: o **estilo de comunicação** (síncrono ou assíncrono) e a **coordenação de operações** que afetam mais de um serviço. A banca costuma cobrar as duas dimensões em conjunto.

Síncrono versus assíncrono

Na comunicação **síncrona**, o serviço chamador envia uma requisição e bloqueia (logicamente) até receber a resposta. O padrão dominante é HTTP/REST; em cenários de alta performance, gRPC. A vantagem é o modelo mental simples: o código parece uma chamada de função normal. A desvantagem é o **acoplamento temporal**: se o serviço B está fora do ar, o serviço A também falha em consequência.

Na comunicação **assíncrona**, o serviço produtor publica uma mensagem em um broker (RabbitMQ, Apache Kafka, AWS SQS) e segue adiante; o consumidor processa quando puder. O acoplamento temporal cai drasticamente: o consumidor pode estar fora do ar por horas e, ao voltar, encontra as mensagens na fila. Isso aumenta resiliência, permite buffer em picos de carga e habilita modelos como event sourcing.

Aspecto	Síncrona (REST/gRPC)	Assíncrona (mensageria)
Acoplamento temporal	Alto (chamador depende do callee disponível)	Baixo (callee processa quando puder)
Modelo mental	Chamada de função (simples)	Eventos e filas (mais cognitivo)
Latência observada pelo cliente	Soma das chamadas downstream	Imediata para o produtor; consumo desacoplado
Casos típicos	Consultas, fluxos transacionais curtos	Notificações, integrações, fluxos longos, ETLs

Saga: consistência sem 2PC

Quando uma operação de negócio afeta vários microserviços (por exemplo, registrar um pedido envolve estoque, pagamento e logística), surge a questão da consistência. Em bancos relacionais usamos transações ACID; em ambientes distribuídos, o **2PC (two-phase commit)** existe, mas tem reputação ruim: ele bloqueia recursos durante a fase de preparação, não tolera bem falhas do coordenador e não escala em sistemas com muitos participantes.

A resposta arquitetural mais difundida é o padrão **Saga**. A ideia: decompõe a operação em uma sequência de **transações locais**, uma em cada serviço; se alguma falha no meio, dispare **transações compensatórias** que desfçam logicamente os passos anteriores. Trocamos consistência forte

(ACID distribuído) por **consistência eventual**: por um curto intervalo, o sistema pode mostrar estados parciais, até as compensações concluírem.

A Saga tem duas variantes principais: a **coreografia**, em que cada serviço reage a eventos publicados pelos demais (descentralizada); e a **orquestração**, em que um orquestrador central conduz a sequência de passos (mais simples de raciocinar, com risco de virar um gargalo). A escolha entre as duas depende do tamanho do fluxo e da visibilidade que o time precisa do estado global da operação.



Repare que, quando uma etapa falha, ativa-se o caminho vermelho de baixo, que percorre os serviços já comitados na ordem inversa, desfazendo cada um por meio de uma transação compensatória local.

Fonte: Prof. Vinicius Borges

Para a prova:

- Comunicação síncrona (REST/gRPC) é fácil de raciocinar, mas tem alto acoplamento temporal.
- Mensageria (RabbitMQ, Kafka) reduz acoplamento e aumenta resiliência, ao custo de consistência eventual.
- Saga substitui 2PC em microsserviços: sequência de transações locais com compensações em caso de falha.
- Duas variantes: coreografia (eventos) e orquestração (orquestrador central).

HORA DE PRATICAR!



(PROF. VINICIUS BORGES / INÉDITA - 2026)

Em arquiteturas de microsserviços, padrões de comunicação assíncrona baseados em mensageria (por exemplo, RabbitMQ ou Apache Kafka) reduzem o acoplamento temporal entre serviços, permitindo que o produtor publique uma mensagem sem precisar aguardar a resposta imediata do consumidor.

Comentários:

Realmente, esse é o ponto central da escolha por mensageria. Como vimos nesta seção, o produtor entrega a mensagem ao broker e segue adiante; o consumidor processa em seu próprio ritmo. Esse desacoplamento temporal é o que torna o sistema mais resiliente a falhas isoladas e capaz de absorver picos de carga sem cascatear a falha.

Gabarito: Certo

API Gateway e Backend For Frontend

Chegamos ao tema central do material. Quando uma aplicação cresce em número de microsserviços e em variedade de clientes (web, mobile, smart TV, parceiros B2B, dispositivos IoT), surge a pergunta: como cada cliente conversa com a malha de serviços? Expor cada microsserviço diretamente para o cliente final é arriscado (segurança, acoplamento, evolução). É aí que entram dois padrões complementares: o **API Gateway** e o **Backend For Frontend (BFF)**.

API Gateway

O **API Gateway** é um componente de infraestrutura posicionado na borda da arquitetura, entre os clientes externos e a malha de microsserviços. Ele funciona como ponto único de entrada e cuida de preocupações transversais que não fazem sentido replicar em cada serviço: autenticação inicial (validação de token, integração com IdP), autorização baseada em escopos, rate limiting, throttling, roteamento, logging, métricas, agregação básica e versionamento de API.

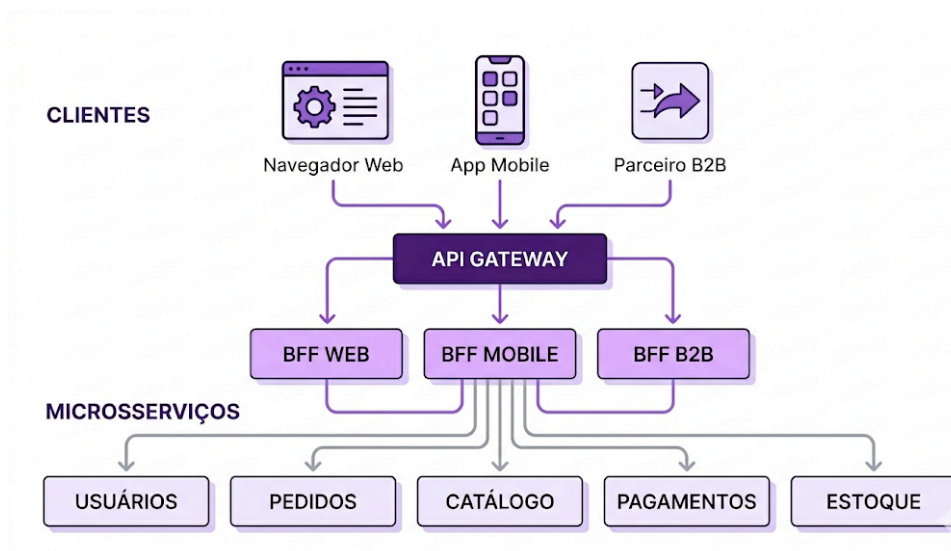
Note que o API Gateway atende a todos os tipos de cliente de forma uniforme. Ele não conhece particularidades de cada experiência. Se uma tela do app mobile precisa combinar dados de cinco microsserviços, o gateway não vai fazer essa agregação específica; ele vai apenas rotear cada chamada e devolver os resultados brutos. Quem se preocupa com a agregação adaptada ao cliente é a próxima camada.

Backend For Frontend (BFF)

O padrão **Backend For Frontend**, descrito por Sam Newman em 2015 (e originário do time da SoundCloud), surge para resolver um problema típico em arquiteturas com vários tipos de cliente: o backend genérico tende a se sobrecarregar de parâmetros condicionais ("se o cliente for mobile, devolva esse formato; se for web, devolva aquele"), e nenhuma equipe se sente dona do contrato.

A solução do BFF é simples na ideia e poderosa na prática: criar uma camada de aplicação por tipo de cliente, em que cada BFF é um backend dedicado àquela experiência. O BFF agrega dados de múltiplos microsserviços downstream, transforma os payloads para o formato ideal daquele cliente, otimiza o número de chamadas (importante para mobile, em que cada round trip custa caro em bateria e latência) e cuida de regras de apresentação específicas.

"Um Backend For Frontend é um pequeno serviço de backend feito sob medida para um único tipo de experiência de usuário. Ele agrega as chamadas downstream necessárias e formata a resposta exatamente para o seu cliente." (Sam Newman, 2015)



Observe que cada cliente conversa com seu próprio BFF; os BFFs compartilham os mesmos microserviços de domínio, mas cada um agrega e formata os dados de modo diferente para a experiência que serve.

Fonte: Prof. Vinicius Borges

Gateway e BFF coexistindo

Na prática, gateway e BFF não competem: eles ocupam camadas diferentes e costumam aparecer juntos. O cliente fala com o API Gateway, que valida token, aplica rate limiting e roteia para o BFF correspondente. O BFF, por sua vez, faz chamadas aos microserviços de domínio e devolve um payload pronto para consumo daquele cliente. Se você quiser memorizar de forma rápida: gateway é infraestrutura transversal, BFF é aplicação por experiência.

Característica	API Gateway	BFF
Camada	Infraestrutura (borda)	Aplicação (entre cliente e domínio)
Quantidade	Tipicamente um (com replicação) para toda a aplicação	Um por tipo de cliente (web, mobile, parceiro, IoT)
Preocupações	Autenticação, rate limiting, roteamento, observabilidade	Agregação de dados, transformação de payload, regras de apresentação
Ciclo de vida	Evolução governada pela área de plataforma	Evolução governada pelo time daquele cliente

Para a prova:

- API Gateway: ponto único de entrada com preocupações transversais (segurança, roteamento, rate limiting).

- BFF: camada por tipo de cliente, com agregação e transformação específicas daquela experiência.
- Os dois coexistem; não são alternativas. Gateway é infraestrutura; BFF é aplicação.
- Originário de Sam Newman e do time da SoundCloud (2015).

HORA DE PRATICAR!



(PROF. VINICIUS BORGES / INÉDITA - 2026)

Sobre a relação entre API Gateway e Backend For Frontend (BFF), assinale a alternativa correta.

- a) API Gateway e BFF são sinônimos; ambos representam a mesma camada de roteamento entre clientes e microsserviços.
- b) O API Gateway implementa a lógica de agregação específica de cada cliente, enquanto o BFF cuida de tarefas transversais.
- c) O API Gateway atua como ponto único de entrada para múltiplos clientes (roteamento, autenticação, rate limiting); o BFF é uma camada por tipo de cliente, com agregações e transformações específicas para aquela experiência.
- d) BFFs são utilizados internamente entre microsserviços, enquanto API Gateways atendem somente parceiros B2B.
- e) O BFF substitui o serviço de descoberta, enquanto o API Gateway substitui o registro de configuração centralizado.

Comentários:

Conforme estudamos nesta seção, gateway e BFF são complementares e ocupam camadas distintas. O gateway é o componente de infraestrutura, responsável por preocupações transversais; o BFF é a camada de aplicação por tipo de cliente, com agregação e transformação dedicadas. As demais alternativas invertem papéis ou introduzem conceitos não relacionados (service discovery, configuração centralizada).

Gabarito: Letra C

Anti-patterns e Service Mesh

Para fechar o material, dois temas que costumam aparecer em provas avançadas: os **anti-patterns** que comprometem arquiteturas de microsserviços (com destaque para os relacionados ao BFF) e o conceito de **Service Mesh**, que vem se consolidando como camada de infraestrutura para resolver problemas transversais de comunicação.

Anti-patterns do BFF

O anti-pattern mais clássico do BFF é a **erosão de responsabilidades**. Como o BFF fica próximo do cliente e tem visão integrada de várias funcionalidades, é tentador colocar nele regras de negócio que deveriam estar nos microsserviços de domínio. Com o tempo, o BFF acumula lógica essencial e vira um novo monolito que toda a aplicação consome. Quando isso acontece, a vantagem original do padrão (camada fina e específica por cliente) se perde.

Outro anti-pattern é manter **regras de agregação idênticas** entre BFFs distintos, replicadas em um repositório compartilhado. Essa prática, na intenção de evitar duplicação, recria o acoplamento que o padrão BFF queria eliminar: passa-se a depender de coordenação entre times sempre que se altera a agregação. O compartilhamento entre BFFs deve ficar restrito a bibliotecas utilitárias (parsing de JWT, clientes HTTP, logging), não a regras de agregação específicas de cliente.

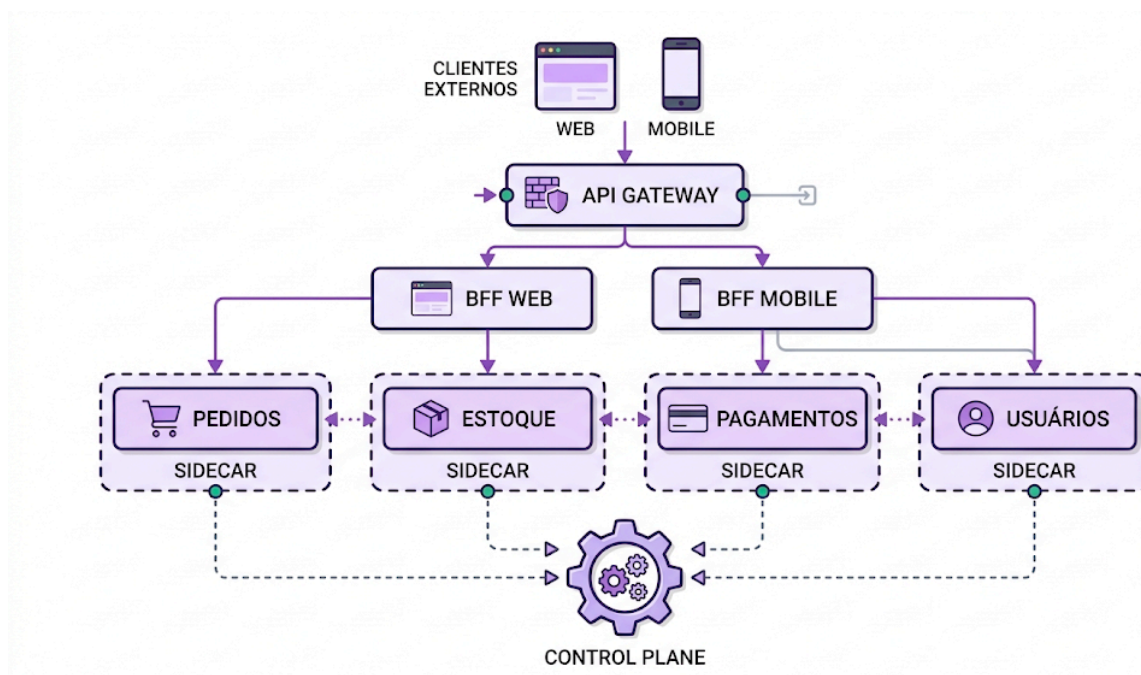
Um terceiro anti-pattern, mais sutil, é confundir BFF com **camada anti-corrupção**. O BFF não é o lugar de "esconder" um microsserviço mal-modelado: se um serviço downstream tem contrato ruim, o problema deve ser resolvido no próprio serviço. Caso contrário, o BFF acumula adaptações que viram dívida técnica.

Service Mesh

A **Service Mesh** é uma camada de infraestrutura para a comunicação entre microsserviços. Em vez de cada serviço implementar bibliotecas próprias de resiliência (circuit breaker, retry, timeout), segurança (mTLS) e observabilidade, essas preocupações são movidas para **proxies sidecars** injetados ao lado de cada instância. O proxy intercepta todo o tráfego de entrada e saída do serviço e aplica as políticas configuradas em um **control plane** central. Soluções de referência são Istio, Linkerd e Consul Connect.

A vantagem prática é que o código de negócio fica limpo: o desenvolvedor não precisa importar bibliotecas específicas de resiliência ou autenticação mútua. As políticas (timeout de 200ms, retry com backoff exponencial, mTLS habilitado, roteamento canário 90/10) são declaradas no control plane e aplicadas uniformemente em todos os serviços, em qualquer linguagem. Isso é particularmente valioso em arquiteturas heterogêneas, em que reimplementar essas preocupações por stack seria custoso.

Componente	Onde fica	Responsabilidade típica
API Gateway	Borda da arquitetura (entrada externa)	Autenticação inicial, rate limiting, roteamento de clientes externos
BFF	Camada de aplicação por cliente	Agregação e transformação específicas para cada experiência
Service Mesh (sidecar)	Ao lado de cada microsserviço (data plane)	mTLS, retries, circuit breaker, observabilidade transversal
Microsserviço de domínio	Núcleo da arquitetura	Regras de negócio e dados de seu contexto delimitado



Repare na separação por responsabilidades: borda externa cuidada pelo gateway, agregação por cliente cuidada pelos BFFs, e comunicação transversal entre serviços internos governada pelos sidecars da mesh, todos orquestrados pelo control plane.

Fonte: Prof. Vinicius Borges

Para a prova:

- Principal anti-pattern do BFF: acumular regras de negócio centrais e virar um novo monolito.
- Compartilhar regras de agregação entre BFFs em repositório único reintroduz o acoplamento que o padrão evita.

- Service Mesh = camada de infraestrutura via sidecars para mTLS, retries, observabilidade e roteamento.
- Mesh não substitui gateway nem BFF; cada um resolve um problema diferente em uma camada diferente.

HORA DE PRATICAR!



(PROF. VINICIUS BORGES / INÉDITA - 2026)

A respeito do papel de uma Service Mesh (por exemplo, Istio ou Linkerd) em arquiteturas de microsserviços, assinale a alternativa correta.

- a) A Service Mesh substitui o BFF como camada de adaptação por cliente, agregando dados de múltiplos serviços.
- b) A Service Mesh exige que as bibliotecas de circuit breaker e retry sejam embarcadas dentro do código de cada microsserviço.
- c) A Service Mesh hospeda os bancos de dados de cada serviço, garantindo isolamento físico dos dados.
- d) A Service Mesh substitui o API Gateway, atendendo diretamente ao tráfego externo dos clientes finais.
- e) A Service Mesh fornece, por meio de sidecars (proxies) ao lado de cada instância, recursos transversais como mTLS, observabilidade, retries, circuit breakers e roteamento, sem exigir mudança no código do serviço.

Comentários:

A letra E descreve corretamente o papel da Service Mesh. Conforme estudamos, o modelo de sidecar é o que move as preocupações transversais para fora do código de negócio: o proxy intercepta o tráfego do serviço e aplica políticas configuradas no control plane. As demais alternativas confundem o papel da mesh com BFF, gateway, banco de dados ou abordagens antigas de bibliotecas embarcadas.

Gabarito: Letra E

QUESTÕES COMENTADAS

HORA DE PRATICAR!



1. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Acerca de arquitetura de microsserviços, julgue o item a seguir: microsserviços são um estilo arquitetural em que uma aplicação é estruturada como um conjunto de pequenos serviços autônomos, cada um executando em seu próprio processo e comunicando-se por meio de mecanismos leves, frequentemente APIs HTTP.

Comentários:

O enunciado reproduz com fidelidade a definição clássica de microsserviços apresentada por Martin Fowler e James Lewis. A ideia central é decompor uma aplicação em serviços pequenos, com responsabilidades bem delimitadas, em que cada serviço roda em processo próprio, mantém autonomia sobre seus dados e expõe um contrato bem definido. A comunicação leve, tipicamente HTTP/REST ou gRPC para casos síncronos e mensageria para casos assíncronos, é parte essencial da definição.

Gabarito: Certo

2. (PROF. VINICIUS BORGES / INÉDITA - 2026)

O padrão Backend For Frontend (BFF), descrito por Sam Newman, surge para resolver qual problema típico em arquiteturas de microsserviços?

- a) A necessidade de fornecer uma camada de API dedicada a cada tipo de cliente (web, mobile, smart TV), evitando que o backend genérico acumule responsabilidades específicas de cada experiência.
- b) A ausência de banco de dados compartilhado entre microsserviços, o que impediria consultas relacionais com joins.
- c) A dificuldade de versionar microsserviços de forma independente em equipes geograficamente distribuídas.
- d) A indisponibilidade de protocolos como gRPC para comunicação interna entre serviços.
- e) A falta de mecanismos nativos de observabilidade em arquiteturas distribuídas.

Comentários:

A motivação original do BFF, conforme Sam Newman, é justamente essa: cada experiência de cliente (web, mobile, smart TV, parceiro B2B) tem necessidades de agregação, formato de payload, tolerância de latência e ciclo de vida próprios. Um backend genérico que atende a todos esses clientes tende a virar uma colcha de retalhos, com endpoints sobrecarregados de parâmetros condicionais. O BFF resolve isso oferecendo uma camada dedicada por tipo de cliente, em que o time daquele cliente mantém autonomia para evoluir o contrato.

Gabarito: Letra A

3. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Em arquiteturas de microsserviços, a independência de implantação implica que cada alteração no contrato de API de um serviço dispara o redeploy automático dos serviços consumidores, garantindo sincronia entre as versões em produção.

Comentários:

Cuidado, pessoal! A independência de implantação significa o contrário do que a assertiva afirma. A meta do estilo arquitetural é justamente permitir que cada serviço seja implantado isoladamente, sem coordenação forçada com os consumidores. Para isso, mudanças no contrato devem ser feitas de forma retrocompatível (versionamento, expansão antes de contração, suporte a múltiplas versões durante a transição). Forçar redeploy automático de consumidores recriaria o acoplamento de implantação típico de monolitos distribuídos, que é exatamente o que se quer evitar.

Gabarito: Errado

4. (PROF. VINICIUS BORGES / INÉDITA - 2026)

A respeito das diferenças entre arquitetura monolítica e arquitetura de microsserviços, assinale a alternativa correta.

- a) Em monolitos modulares, escalar uma funcionalidade específica é mais econômico do que em microsserviços, pois basta replicar apenas o módulo desejado.
- b) Microsserviços permitem que diferentes serviços usem stacks tecnológicas distintas (banco, linguagem, framework), enquanto o monolito tende a impor uma única stack para toda a aplicação.
- c) A latência média entre componentes de um monolito tende a ser maior do que entre microsserviços, pois chamadas in-process são mais lentas que chamadas de rede.

- d) Microserviços eliminam a necessidade de testes de integração, já que cada serviço é testado isoladamente.
- e) Em arquiteturas monolíticas, o time precisa coordenar deploys por serviço, enquanto em microserviços o deploy é único para a aplicação inteira.

Comentários:

A heterogeneidade tecnológica é uma das vantagens mais citadas dos microserviços: como cada serviço tem seu próprio processo e banco, ele pode adotar a stack que melhor se ajusta ao seu domínio (Node para um BFF de UI, Go para um serviço de baixa latência, Python para um pipeline de dados). O monolito, por construção, compartilha runtime e dependências, o que dificulta misturar stacks.

Gabarito: Letra B

5. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Em arquiteturas de microserviços, padrões de comunicação assíncrona baseados em mensageria (por exemplo, RabbitMQ ou Apache Kafka) reduzem o acoplamento temporal entre serviços, permitindo que o produtor publique uma mensagem sem precisar aguardar a resposta imediata do consumidor.

Comentários:

Realmente, esse é um dos motivos centrais para preferir mensageria em fluxos que não exigem resposta síncrona. O acoplamento temporal acontece quando o produtor depende do consumidor estar disponível naquele exato instante para concluir sua operação. Com brokers como RabbitMQ ou Kafka, o produtor publica em um tópico ou fila e segue adiante; o consumidor processa quando puder. Isso aumenta resiliência (consumidor pode estar fora do ar temporariamente sem derrubar o produtor), permite buffer em picos de carga e habilita modelos de processamento como event-driven e event sourcing.

Gabarito: Certo

6. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Sobre a relação entre API Gateway e Backend For Frontend (BFF) em arquiteturas de microserviços, assinale a alternativa correta.

- a) API Gateway e BFF são sinônimos; ambos representam a mesma camada de roteamento entre clientes e microserviços.

- b) O API Gateway implementa a lógica de agregação específica de cada cliente, enquanto o BFF se limita a tarefas transversais como autenticação e rate limiting.
- c) O API Gateway atua como ponto único de entrada para múltiplos clientes, cuidando de tarefas transversais (roteamento, autenticação, rate limiting); o BFF é uma camada por tipo de cliente, com agregações e transformações específicas para aquela experiência.
- d) BFFs são utilizados internamente entre microserviços, enquanto API Gateways atendem somente parceiros B2B autenticados.
- e) O BFF substitui o serviço de descoberta (service discovery), enquanto o API Gateway substitui o registro de configuração centralizado.

Comentários:

A distinção correta é essa: o API Gateway é um componente de infraestrutura, posicionado na borda, que centraliza preocupações transversais (autenticação inicial, rate limiting, observabilidade, roteamento bruto). Ele atende todos os clientes igualmente. Já o BFF é uma camada de aplicação, com um BFF por tipo de cliente, em que cabe a agregação de dados de múltiplos microserviços downstream e a adaptação do payload para o que aquele cliente precisa. Os dois costumam coexistir em uma mesma arquitetura: o cliente fala com o API Gateway, que roteia para o BFF correspondente, que por sua vez agrega serviços internos.

Gabarito: Letra C

7. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Em uma arquitetura com múltiplos BFFs (um para web, um para mobile e um para parceiros B2B), recomenda-se manter um único repositório de código compartilhado entre eles, de modo que regras de agregação sejam idênticas em todos os clientes e qualquer alteração em um BFF se propague para os demais.

Comentários:

Nada disso! A essência do padrão BFF é justamente o oposto: cada cliente tem sua própria camada de agregação, com autonomia para evoluir o contrato sem coordenação forçada com os demais. Se as regras de agregação forem idênticas e replicadas em um repositório único, perde-se a flexibilidade que motivou o padrão e cria-se um ponto de coordenação que recria o acoplamento de monolito. O compartilhamento entre BFFs deve ocorrer apenas em bibliotecas utilitárias (parsing de JWT, clientes HTTP, logging), não em regras de agregação específicas de cliente.

Gabarito: Errado

8. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Em arquiteturas com Backend For Frontend, qual situação caracteriza um anti-pattern relevante?

- a) Cada BFF possui seu próprio time, com autonomia para evoluir o contrato em conjunto com o cliente correspondente.
- b) O BFF agrega dados de múltiplos microsserviços downstream antes de devolver a resposta para o cliente.
- c) O BFF aplica autenticação delegando a verificação de token a um serviço de identidade centralizado.
- d) O BFF passa a concentrar regras de negócio centrais do domínio, transformando-se gradualmente em um novo monolito que toda a aplicação consome.
- e) Cada tipo de cliente (web, mobile, IoT) possui um BFF dedicado, com agregações específicas para sua experiência.

Comentários:

O anti-pattern clássico do BFF é a chamada erosão de responsabilidades: como o BFF fica perto do cliente e tem visão integrada de várias funcionalidades, é tentador colocar nele regras de negócio que pertencem aos microsserviços de domínio. Quando isso se acumula, o BFF deixa de ser uma fina camada de adaptação e vira um novo gargalo, com lógica essencial duplicada ou monopolizada.

Gabarito: Letra D

9. (PROF. VINICIUS BORGES / INÉDITA - 2026)

O padrão Saga, em microsserviços, é uma alternativa às transações distribuídas (2PC) que coordena uma sequência de transações locais nos serviços envolvidos, compensando passos anteriores caso uma etapa falhe; com isso, abre-se mão da consistência forte em favor da consistência eventual.

Comentários:

Conforme estudamos, o padrão Saga é a resposta arquitetural mais difundida para manter consistência em operações que tocam vários microsserviços. Em vez de bloquear recursos com um coordenador 2PC (que não escala bem em ambientes distribuídos), a Saga executa uma sequência de transações locais, cada uma comitadas em seu próprio serviço. Se uma etapa falha, são acionadas transações compensatórias que desfazem logicamente os passos anteriores. O custo dessa escolha é trocar consistência forte (ACID distribuído) por consistência eventual: por um curto intervalo, o sistema pode mostrar estados parcialmente atualizados, até as compensações concluírem. Existem

duas variantes principais: coreografia (cada serviço reage a eventos) e orquestração (um coordenador central conduz o fluxo).

Gabarito: Certo

10.(PROF. VINICIUS BORGES / INÉDITA - 2026)

A respeito do papel de uma Service Mesh (por exemplo, Istio ou Linkerd) em arquiteturas de microsserviços, assinale a alternativa correta.

- a) A Service Mesh substitui o BFF como camada de adaptação por cliente, agregando dados de múltiplos serviços para cada tipo de tela.
- b) A Service Mesh exige que as bibliotecas de circuit breaker e retry sejam embarcadas dentro do código de cada microsserviço, dispensando proxies externos.
- c) A Service Mesh é responsável por hospedar os bancos de dados de cada serviço, garantindo isolamento físico dos dados.
- d) A Service Mesh substitui o API Gateway, atendendo diretamente ao tráfego externo dos clientes finais.
- e) A Service Mesh fornece, por meio de sidecars (proxies) ao lado de cada instância, recursos transversais de comunicação como mTLS, observabilidade, retries, circuit breakers e roteamento, sem exigir mudança no código do serviço.

Comentários:

A Service Mesh resolve um problema clássico de microsserviços: muitos times reimplementavam, em cada serviço, as mesmas preocupações de comunicação (resiliência, segurança, observabilidade). A solução é mover essas preocupações para uma camada de infraestrutura, em que um proxy (sidecar) é injetado junto a cada instância do serviço. Esse proxy intercepta o tráfego de entrada e saída do serviço e aplica as políticas configuradas no control plane (Istio, Linkerd). A vantagem é que o código de negócio fica limpo: o desenvolvedor não precisa escrever circuit breaker, retry policy ou mTLS dentro da aplicação.

Gabarito: Letra E

GABARITO

GABARITO



1. Certo
2. Letra A
3. Errado
4. Letra B

5. Certo
6. Letra C
7. Errado
8. Letra D

9. Certo
10. Letra E

LISTA DE QUESTÕES

HORA DE PRATICAR!



1. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Acerca de arquitetura de microsserviços, julgue o item a seguir: microsserviços são um estilo arquitetural em que uma aplicação é estruturada como um conjunto de pequenos serviços autônomos, cada um executando em seu próprio processo e comunicando-se por meio de mecanismos leves, frequentemente APIs HTTP.

2. (PROF. VINICIUS BORGES / INÉDITA - 2026)

O padrão Backend For Frontend (BFF), descrito por Sam Newman, surge para resolver qual problema típico em arquiteturas de microsserviços?

- a) A necessidade de fornecer uma camada de API dedicada a cada tipo de cliente (web, mobile, smart TV), evitando que o backend genérico acumule responsabilidades específicas de cada experiência.
- b) A ausência de banco de dados compartilhado entre microsserviços, o que impediria consultas relacionais com joins.
- c) A dificuldade de versionar microsserviços de forma independente em equipes geograficamente distribuídas.
- d) A indisponibilidade de protocolos como gRPC para comunicação interna entre serviços.
- e) A falta de mecanismos nativos de observabilidade em arquiteturas distribuídas.

3. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Em arquiteturas de microsserviços, a independência de implantação implica que cada alteração no contrato de API de um serviço dispara o redeploy automático dos serviços consumidores, garantindo sincronia entre as versões em produção.

4. (PROF. VINICIUS BORGES / INÉDITA - 2026)

A respeito das diferenças entre arquitetura monolítica e arquitetura de microsserviços, assinale a alternativa correta.

- a) Em monolitos modulares, escalar uma funcionalidade específica é mais econômico do que em microsserviços, pois basta replicar apenas o módulo desejado.
- b) Microsserviços permitem que diferentes serviços usem stacks tecnológicas distintas (banco, linguagem, framework), enquanto o monolito tende a impor uma única stack para toda a aplicação.
- c) A latência média entre componentes de um monolito tende a ser maior do que entre microsserviços, pois chamadas in-process são mais lentas que chamadas de rede.
- d) Microsserviços eliminam a necessidade de testes de integração, já que cada serviço é testado isoladamente.
- e) Em arquiteturas monolíticas, o time precisa coordenar deploys por serviço, enquanto em microsserviços o deploy é único para a aplicação inteira.

5. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Em arquiteturas de microsserviços, padrões de comunicação assíncrona baseados em mensageria (por exemplo, RabbitMQ ou Apache Kafka) reduzem o acoplamento temporal entre serviços, permitindo que o produtor publique uma mensagem sem precisar aguardar a resposta imediata do consumidor.

6. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Sobre a relação entre API Gateway e Backend For Frontend (BFF) em arquiteturas de microsserviços, assinale a alternativa correta.

- a) API Gateway e BFF são sinônimos; ambos representam a mesma camada de roteamento entre clientes e microsserviços.
- b) O API Gateway implementa a lógica de agregação específica de cada cliente, enquanto o BFF se limita a tarefas transversais como autenticação e rate limiting.
- c) O API Gateway atua como ponto único de entrada para múltiplos clientes, cuidando de tarefas transversais (roteamento, autenticação, rate limiting); o BFF é uma camada por tipo de cliente, com agregações e transformações específicas para aquela experiência.
- d) BFFs são utilizados internamente entre microsserviços, enquanto API Gateways atendem somente parceiros B2B autenticados.
- e) O BFF substitui o serviço de descoberta (service discovery), enquanto o API Gateway substitui o registro de configuração centralizado.

7. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Em uma arquitetura com múltiplos BFFs (um para web, um para mobile e um para parceiros B2B), recomenda-se manter um único repositório de código compartilhado entre eles, de modo que regras de agregação sejam idênticas em todos os clientes e qualquer alteração em um BFF se propague para os demais.

8. (PROF. VINICIUS BORGES / INÉDITA - 2026)

Em arquiteturas com Backend For Frontend, qual situação caracteriza um anti-pattern relevante?

- a) Cada BFF possui seu próprio time, com autonomia para evoluir o contrato em conjunto com o cliente correspondente.
- b) O BFF agrega dados de múltiplos microsserviços downstream antes de devolver a resposta para o cliente.
- c) O BFF aplica autenticação delegando a verificação de token a um serviço de identidade centralizado.
- d) O BFF passa a concentrar regras de negócio centrais do domínio, transformando-se gradualmente em um novo monolito que toda a aplicação consome.
- e) Cada tipo de cliente (web, mobile, IoT) possui um BFF dedicado, com agregações específicas para sua experiência.

9. (PROF. VINICIUS BORGES / INÉDITA - 2026)

O padrão Saga, em microsserviços, é uma alternativa às transações distribuídas (2PC) que coordena uma sequência de transações locais nos serviços envolvidos, compensando passos anteriores caso uma etapa falhe; com isso, abre-se mão da consistência forte em favor da consistência eventual.

10. (PROF. VINICIUS BORGES / INÉDITA - 2026)

A respeito do papel de uma Service Mesh (por exemplo, Istio ou Linkerd) em arquiteturas de microsserviços, assinale a alternativa correta.

- a) A Service Mesh substitui o BFF como camada de adaptação por cliente, agregando dados de múltiplos serviços para cada tipo de tela.
- b) A Service Mesh exige que as bibliotecas de circuit breaker e retry sejam embarcadas dentro do código de cada microsserviço, dispensando proxies externos.

- c) A Service Mesh é responsável por hospedar os bancos de dados de cada serviço, garantindo isolamento físico dos dados.
- d) A Service Mesh substitui o API Gateway, atendendo diretamente ao tráfego externo dos clientes finais.
- e) A Service Mesh fornece, por meio de sidecars (proxies) ao lado de cada instância, recursos transversais de comunicação como mTLS, observabilidade, retries, circuit breakers e roteamento, sem exigir mudança no código do serviço.

GABARITO

GABARITO



1. Certo
2. Letra A
3. Errado
4. Letra B

5. Certo
6. Letra C
7. Errado
8. Letra D

9. Certo
10. Letra E

PADRÕES DE INTEGRAÇÃO

Conceitos Básicos

Pessoal, antes de a gente falar do "Gateway" (o Porteiro), vamos "destrinchar" rapidinho o que é a "API". Para isso, vamos fazer uma analogia:

Pense num restaurante. Você (o Cliente) está na mesa e quer comer. A Cozinha (o Sistema Backend) é onde a comida está sendo feita, mas você não pode simplesmente entrar na cozinha e pegar sua comida.

Você precisa de um intermediário: o **Garçom**.

- Você faz um **pedido** (Requisição) ao garçom, usando um formato que ele entende (o cardápio).
- O Garçom leva o pedido para a Cozinha.
- A Cozinha prepara.
- O Garçom traz a **comida** (Resposta) para você.

A **API** (Application Programming Interface, ou Interface de Programação de Aplicações) é o garçom. É um **conjunto de regras e padrões** que permite que sistemas diferentes conversem entre si (o cliente e o backend) de forma organizada, sem que um precise saber como o outro funciona por dentro.

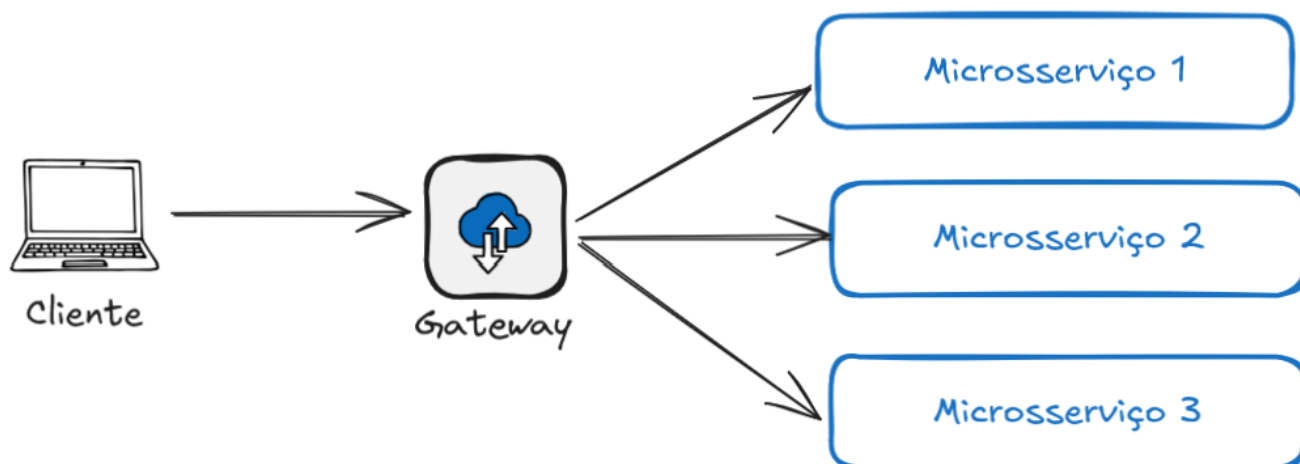
Beleza. Agora, imaginem uma arquitetura de microsserviços. Em vez de um "blocão" de código (monolito) com uma cozinha só, a gente tem **dezenas ou até centenas de "cozinhas"** pequenas e independentes: a cozinha das bebidas (serviço de usuário), a cozinha das entradas (serviço de produto), a cozinha dos pratos principais (serviço de pagamento), entre outras.

Imagina um cliente tentando pedir comida nesse restaurante. Ele teria que chamar cada garçom de cada cozinha, independentemente, para ser atendido.

API Gateway (Definição)

Para resolver essa bagunça, a gente coloca um "Porteiro" na entrada do restaurante. Esse cara é o API Gateway.

O API Gateway é um padrão de software que atua como um **ponto de entrada único** para todas as requisições dos clientes. O app cliente não fala mais com 10 garçons; ele fala com *um* lugar só: o Gateway. O Gateway, então, se encarrega de **rotear a requisição** para o garçom (microsserviço) apropriado lá dentro.



Fonte: Prof. Vinicius Borges

Sua função mais fundamental e básica é exatamente essa: atuar como um **Proxy Reverso** e fazer o **Roteamento de Gateway**.

Gateway vs. Proxy Reverso (Uma Dúvida Clássica)

Muita gente confunde, e essa é uma distinção que já apareceu em provas. Vamos esclarecer:

- **Proxy Reverso (Genérico):** É um componente de rede que atua como **intermediário entre clientes externos e servidores internos**. Ele "oculta" a infraestrutura interna e geralmente oferece funções como **balanceamento de carga**, **cache** e **terminação SSL**. É um conceito mais amplo.
- **API Gateway (Especializado):** Todo API Gateway é, em essência, um Proxy Reverso. Porém, ele é um **proxy especializado em gerenciamento de APIs**. Ele faz tudo que um proxy reverso faz, mas adiciona uma camada de inteligência específica para APIs: **roteamento avançado** baseado em API, **segurança (validação de token)**, **limitação de taxa** (rate limiting), **monitoramento**, etc.

Em outras palavras, todo API Gateway é um **proxy reverso**, **nem todo proxy reverso é um API gateway**.

Funções essenciais

O Gateway tem uma lista de tarefas, conhecidas como "**funcionalidades transversais**" (cross-cutting concerns), que são funções necessárias em vários serviços, mas não fazem parte da lógica de negócios principal de nenhum serviço específico:

- **Segurança Centralizada:** Essa é uma das maiores vantagens. Você implementa a segurança da API em um único ponto. O Gateway verifica o token de autenticação, se o usuário tem autorização, etc.
 - Cuidado! Embora o Gateway centralize a *primeira camada* de segurança, isso **não substitui** a necessidade de os microsserviços internos também terem mecanismos de segurança. Isso é o princípio da "Defesa em Profundidade" ou "Zero Trust".
- **Roteamento e Configuração:** O Gateway precisa saber para onde mandar a chamada. Para isso, na configuração dele, você precisa especificar não só a **rota** (o caminho, ex: `/api/fiscalizacao`) mas também o **método HTTP** (GET, POST, etc.) que o backend suporta.
- **Monitoramento e Logging:** Justamente por ser o ponto de entrada único, o Gateway é o local *ideal* para centralizar o monitoramento e o logging.
- **Rate Limiting:** O gateway pode aplicar uma limitação de taxa de consumo para evitar abusos por parte do usuário.

Padrões Avançados de Gateway

Aqui cabe mencionar alguns termos que já apareceram em provas:

- **Gateway Aggregation (Agregação):** Lembra do cliente que precisava fazer 3 chamadas para garçons diferentes? Com esse padrão, o cliente faz *uma única chamada* ao Gateway (ex: GET `/meu-dashboard`). O Gateway, então, chama os 3 microsserviços internos, "agrega" (junta) as respostas e devolve uma resposta única. A principal consequência é que o Gateway **reduz o número de chamadas externas que se é necessário fazer**.
- **Backend for Frontends (BFF):** E se o seu app Mobile precisar de dados diferentes do seu site Web? O mobile quer 2 campos; o web quer 20. Em vez de um Gateway "monolítico" que tenta servir a todos, você cria um Gateway específico *para cada frontend*. Teremos um "BFF-Mobile" e um "BFF-Web". É um padrão de implementação de Gateway focado na experiência do cliente.
- **Mediação e Tradução:** O Gateway pode atuar como um tradutor. Se o seu cliente fala JSON, mas o microsserviço interno é legado e só entende XML, o Gateway pode fazer essa *transformação*. Ele também pode fazer *tradução de protocolo* (ex: receber uma chamada HTTP e repassar como gRPC para o backend).

Gateway vs Service Mesh

Não confunda! O **API Gateway** gerencia o tráfego **Norte-Sul** (do mundo externo para dentro da aplicação). O **Service Mesh** gerencia o tráfego **Leste-Oeste** (a comunicação interna, entre os microsserviços).

Risco: Ponto Único de Falha (SPOF)

Por fim, galera, cabe uma reflexão sobre o principal risco: se *tudo* passa pelo Gateway, o que acontece se ele cair? Ninguém mais usa o sistema.

A adoção de um API Gateway *introduz* sim um potencial Ponto Único de Falha (SPOF). Para mitigar (diminui) esse risco, nunca rodamos uma instância só do Gateway; usamos um sistema com alta disponibilidade, que possui várias instâncias rodando em paralelo com um balanceador de carga.

LISTA DE QUESTÕES

1. (Ano: 2024 Banca: IMPARH Órgão: Prefeitura de Fortaleza - CE Prova: IMPARH - 2024 - Prefeitura de Fortaleza - CE - Analista de Regulação - Ciências da Computação)

Qual componente é descrito pela seguinte definição?

"Um padrão de software amplamente utilizado em arquiteturas de microsserviços, atuando como um ponto de entrada único para todas as requisições dos clientes e roteando essas requisições para os microsserviços apropriados."

- A) Reverse Proxy.
- B) Load Balancer.
- C) Service Mesh.
- D) API Gateway.

Comentários:

Pessoal, essa primeira é a definição clássica do que é um API Gateway. Ele funciona como um "ponto de entrada único" da sua arquitetura. O ponto-chave aqui é a frase: "em arquiteturas de microsserviços". Em vez de o cliente (app mobile, site) ter que saber o endereço de 10 microsserviços diferentes, ele fala **com um lugar só**: o API Gateway. O Gateway, então, se encarrega de rotear a requisição para o microsserviço apropriado.

Gabarito: D

2. (Ano: 2024 Banca: COMPERVE - UFRN Órgão: UFRN Prova: COMPERVE - UFRN - 2024 - UFRN - Técnico em Tecnologia da Informação (Suporte e Redes))

O serviço atua como um intermediário entre os clientes externos e os servidores internos, recebendo as requisições dos clientes e as encaminhando para os servidores de fato. Com esse serviço, é possível ocultar a infraestrutura do servidor, oferecer armazenamento em cache, aprimorar a segurança e possibilitar um controle mais granular sobre as requisições.

Esse serviço descrito é o:

- A) Proxy reverso.
- B) Load balancer.
- C) API Gateway.
- D) NAT - Network Address Translation.

Comentários:

Galera, cuidado.

Essa questão deu uma definição mais genérica: "intermediário entre clientes externos e servidores internos", "ocultar a infraestrutura", "armazenamento em cache", "aprimorar a segurança". Essas são as funções clássicas de um Proxy Reverso.

Um API Gateway (Alternativa C) é um **tipo de proxy reverso**, mas ele é **especializado**, com mais funções (como **gerenciamento de API**, **limitação de taxa**, entre outros). Como a descrição da questão é mais ampla e se encaixa perfeitamente na definição de Proxy Reverso, essa é a melhor resposta.

Lembre-se disso:

- Todo API Gateway é um proxy reverso
- Nem todo proxy reverso é um API Gateway

Gabarito: A

3. (Ano: 2024 Banca: CESPE / CEBRASPE Órgão: CAU-BR Prova: CESPE / CEBRASPE - 2024 - CAU-BR - Analista de Infraestrutura de Tecnologia da Informação)

Com relação a barramento de serviços corporativos (EBS), e dos aspectos de segurança e versatilidade do API Gateway, julgue o item subsequente.

Uma das vantagens de se usar um API Gateway é que ele permite implementar a segurança da API em um único ponto, em vez de em cada serviço individual.

Comentários:

À primeira vista, a afirmação parece corretíssima. A de um API Gateway é que ele serve para **centralizar "tarefas transversais"** (cross-cutting concerns), e o principal exemplo é a **segurança (autenticação e autorização)**. Em vez de cada microserviço ter que validar um token JWT, o Gateway faz isso na entrada.

Então, por que estaria errado?

A "casca de banana" está na expressão: "...em vez de em cada serviço individual."

Em uma arquitetura de segurança moderna, especialmente seguindo princípios como **Zero Trust (Confiança Zero)** ou **Defense in Depth (Defesa em Profundidade)**, o fato de o Gateway ter validado a segurança **não elimina a responsabilidade** do microserviço individual **de também se validar**.

Pense assim: O Gateway é o segurança na porta principal do prédio. Ele barra quem não tem autorização. Mas isso não significa que a porta do seu apartamento deva ficar destrancada. O

microserviço (o apartamento) também deve validar quem está tentando acessá-lo, mesmo que a chamada venha de "dentro" do prédio (como de outro microserviço).

Portanto, o API Gateway centraliza a primeira camada de validação de segurança na borda (no ponto de entrada), o que é sim uma vantagem enorme. Mas ele **não implementa a segurança em substituição** ("em vez de") aos serviços individuais, que ainda devem ser capazes de se proteger.

A banca foi extremamente técnica e pedante aqui, mas a afirmativa está errada por sugerir que a segurança nos serviços individuais deixa de existir.

Gabarito: Errado

4. (Ano: 2023 Banca: CESPE / CEBRASPE Órgão: SEFIN de Fortaleza - CE Prova: CESPE / CEBRASPE - 2023 - SEFIN de Fortaleza - CE - Analista Fazendário Municipal - Área de Conhecimento: Ciência da Computação, Informática/Processamento de Dados)

Acerca de API Gateway, julgue o próximo item.

Considere-se que seja necessário criar uma API Gateway para um serviço back-end que responde a solicitações HTTP na rota /api/fiscalização. Nessa situação hipotética, na etapa de configuração da rota para o serviço de back-end, é necessário especificar, na API Gateway, o método HTTP (GET, POST etc.) que a aplicação back-end suporta, para que a integração funcione corretamente.

Comentários:

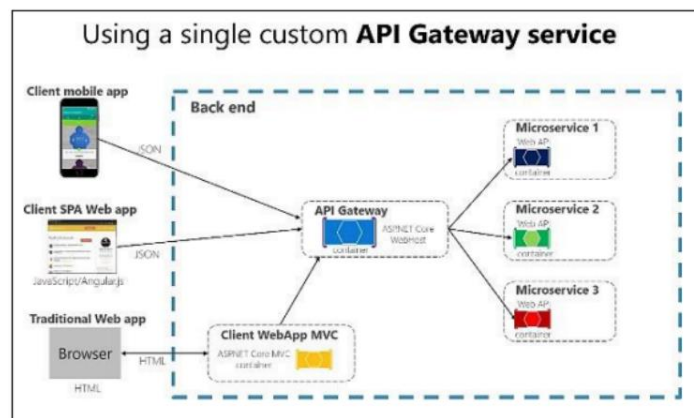
O API Gateway não faz mágica, ele precisa ser configurado. Ele funciona como um roteador inteligente. Para ele saber que uma requisição GET para a rota /api/fiscalizacao deve ir para o Microserviço A, e uma requisição POST para a mesma rota deva ir para o Microserviço B, você **precisa especificar isso na configuração dele**. A rota no Gateway é definida pela **combinação do caminho** (a rota em si) e pelo **método HTTP** (GET, POST, PUT, etc.). Se o método não for especificado ou for diferente do que o back-end suporta, a integração não vai funcionar.

Gabarito: Certo

5. (Ano: 2024 Banca: Instituto Access Órgão: Banestes Prova: Instituto Access - 2024 - Banestes - Analista em Tecnologia da Informação - Segurança da Informação)

A Figura mostra como um Gateway de API personalizado pode se encaixar em uma arquitetura simplificada baseada em microserviço com apenas alguns microserviços. Um Gateway de API pode oferecer vários recursos. Dependendo do produto, ele pode oferecer recursos mais avançados ou mais simples, no entanto, os recursos.

Marque a alternativa que apresenta corretamente, uma das mais importantes e fundamentais para qualquer Gateway de API padrões de design.



- A) Proxy reverso ou roteamento de gateway.
- B) Porta TCP diferente para cada microserviço.
- C) Balanceador de carga e ADC.
- D) Hosts descarregando terminação SSL.

Comentários:

A banca quer saber qual é a característica mais fundamental de um API Gateway. A função mais básica, o "feijão com arroz" de qualquer API Gateway, é **atuar como um Proxy Reverso/fazer o Roteamento de Gateway**. Ou seja, ele recebe a chamada do cliente e a encaminha (roteia) para o microserviço de back-end correto.

As outras opções, como balanceamento de carga ou descarregar terminação SSL, são recursos muito comuns e importantes que um Gateway pode oferecer, mas a sua **função essencial e primária é o roteamento/proxy reverso**.

Gabarito: A

6. (Ano: 2024 Banca: CESPE / CEBRASPE Órgão: SEPLAG-CE Prova: CESPE / CEBRASPE - 2024 - SEPLAG-CE - Analista de Gestão Pública - Área de Atuação: Ciência da Computação) Acerca de gerenciamento de API, de RESTful e de ITIL 4, julgue o item subsequente.

Um API gateway atua como um ponto central de entrada para várias APIs e desempenha um papel importante na simplificação da gestão de tráfego, autenticação, autorização e monitoramento das chamadas de API.

Comentários:

Essa afirmativa é um ótimo resumo das vantagens e funções do API Gateway:

- Ele é o ponto central de entrada;
- Ele simplifica a gestão de tráfego (roteamento);

- Ele cuida da segurança (autenticação e autorização);
- Ele permite o monitoramento das chamadas.

Gabarito: Certo

7. (Ano: 2025 Banca: CESPE / CEBRASPE Órgão: MPE-CE Prova: CESPE / CEBRASPE - 2025 - MPE-CE - Analista Ministerial - Especialidade: Ciências da Computação)

Julgue o item a seguir, referente a ferramentas de integração assíncrona, microserviços, arquitetura orientada a serviços e gerenciamento de serviços.

Um API gateway é responsável pelo armazenamento de dados temporários, e, apesar de executar várias tarefas transversais, não realiza algumas funções, tais como terminação de SSL, TLS mútuo e limitação de taxa.

Comentários:

O API Gateway é exatamente o lugar ideal para realizar essas "tarefas transversais" (cross-cutting concerns).

Funções como **limitação de taxa (rate limiting)**, para evitar abusos da API, e **terminação de SSL** (o cliente fala HTTPS com o gateway, e o gateway pode falar HTTP com o microserviço interno, simplificando o back-end) são **recursos fundamentais de implementações modernas** de API Gateway. A afirmação diz que ele não realiza essas funções, o que torna o item errado.

Gabarito: Errado

8. (Ano: 2024 Banca: CESPE / CEBRASPE Órgão: MPO Prova: CESPE / CEBRASPE - 2024 - MPO - Analista de Planejamento e Orçamento - Especialidade: Desenvolvimento de Sistemas Orçamentários)

A respeito de arquitetura de aplicações, julgue o próximo item.

Um gateway de API funciona como um portfólio de serviços, em que o usuário escolhe e implementa a chamada do serviço necessário.

Comentários:

Essa é uma questão bem sutil, então vamos separar os papéis:

- O API Gateway: Esse é o componente de **runtime**. É a "porta da frente", o "ponto de entrada único". Ele executa as políticas, roteia o tráfego, autentica o usuário, etc. Você não "escolhe" nada no Gateway; você envia a chamada para ele.

- O API Portal / Swagger UI: Esse sim! Este é o componente que pega a especificação (o Swagger) e a apresenta de forma bonita, interativa. É uma "vitrine", um "catálogo". É aqui que o "usuário" (o desenvolvedor) entra, navega, vê o portfólio de serviços, escolhe o que precisa, lê a documentação e, muitas vezes, pode até testar e ver como implementar a chamada.

Portanto, a banca atribuiu ao API Gateway uma função que, na verdade, é do API Portal (que usa o Swagger para gerar essa visualização). O Gateway é o executor da chamada, não o catálogo onde você a escolhe.

Gabarito: Errado

9. (Ano: 2025 Banca: FGV Órgão: DPE-RO Prova: FGV - 2025 - DPE-RO - Analista Programador - Classe B, adaptada)

Um *API Gateway* atua como um *proxy* reverso, ficando entre os aplicativos cliente e os microsserviços, fornecendo um único ponto de entrada para todas as requisições de clientes a um conjunto de serviços.

Comentários:

O API Gateway é um tipo especializado de **Proxy Reverso**. Ele fica no meio do caminho (entre o cliente e os microsserviços) e serve como **ponto de entrada único** para todas as requisições de clientes a um conjunto de serviços.

Gabarito: Certo

10. (Ano: 2025 Banca: FCC Órgão: TRT - 6ª Região (PE) Prova: FCC - 2025 - TRT - 6ª Região (PE) - Analista Judiciário - Área Apoio Especializado - Especialidade: Tecnologia da Informação)

Ao usar o Gateway API com o Service Discovery no contexto da PDPJ-Br, a principal consequência para o frontend em uma aplicação baseada em microsserviços é que o

- A) Gateway API diminui a necessidade de autenticação, uma vez que o Service Discovery é integrado ao SSO.
- B) frontend deve configurar manualmente os IPs e portas dos serviços disponíveis na plataforma.
- C) frontend se torna o responsável pela chamada direta de cada microsserviço individualmente.
- D) Gateway API reduz o número de chamadas externas que o frontend precisa fazer, tornando a comunicação mais eficiente.
- E) frontend deve gerenciar manualmente as atualizações de endpoint de cada serviço individual registrado.

Comentários:

Essa é uma ótima questão! Qual seria consequência para o frontend (o app do cliente)?

Imagine que uma tela no app precisa de dados que estão em 3 microsserviços diferentes.

- Sem Gateway: O frontend teria que fazer 3 chamadas de rede diferentes.
- Com Gateway: O frontend faz uma **única chamada para o Gateway** (ex: GET /meu-dashboard).
 - O Gateway, lá no back-end, é quem se vira para chamar os 3 microsserviços e juntar a resposta (um padrão chamado Gateway Aggregation).

Portanto, o Gateway **reduz o número de chamadas externas** que o frontend precisa fazer, tornando a comunicação mais eficiente.

Gabarito: D

11.(PROF. VINICIUS BORGES / INÉDITA - 2025)

Em arquiteturas de microsserviços, é comum que diferentes clientes (ex: um app mobile e uma aplicação web desktop) tenham necessidades distintas de consumo de dados. O padrão de API Gateway que endereça essa necessidade, provendo endpoints otimizados para cada tipo de cliente, é conhecido como:

- A) Gateway Aggregation
- B) Backend for Frontends (BFF)
- C) Service Mesh
- D) Proxy Reverso Adaptativo
- E) Serverless

Comentários:

Vamos pensar no cenário: o app mobile precisa de 2 campos, o app web precisa de 10 campos. Se eu usar um Gateway só, ou eu mando dados demais pro mobile (gastando banda) ou crio um endpoint específico pra ele. Para organizar isso, surgiu o padrão **BFF (Backend for Frontends)**. A ideia é ter **um Gateway específico para cada tipo de frontend**. Teremos um "API Gateway do Mobile" e um "API Gateway da Web". É a definição exata do padrão.

Gabarito: B

12.(PROF. VINICIUS BORGES / INÉDITA - 2025)

Um API Gateway e um Service Mesh são componentes que desempenham funções idênticas, sendo responsáveis primariamente pela comunicação Leste-Oeste (interna, entre microsserviços).

Comentários:

Nada disso! Grave isso: **API Gateway** gerencia o tráfego "**Norte-Sul**", ou seja, de *fora* da sua arquitetura (o cliente) para *dentro* dela. O **Service Mesh** gerencia o tráfego "**Leste-Oeste**", ou

seja, a comunicação *interna*, de um microserviço para outro microserviço. Embora ambos possam fazer roteamento, descoberta de serviço e etc., eles atuam em perímetros diferentes.

Gabarito: E

13.(PROF. VINICIUS BORGES / INÉDITA - 2025)

O padrão *Gateway Aggregation* refere-se à capacidade do API Gateway de receber uma única requisição do cliente e, internamente, orquestrar chamadas a múltiplos microserviços, consolidando as respostas antes de retorná-las ao cliente.

Comentários:

Essa é a definição do padrão **Gateway Aggregation** (Agregação no Gateway). Pense numa tela de "Meu Perfil" do usuário: ela precisa mostrar dados do "Serviço de Usuário", do "Serviço de Pedidos" e do "Serviço de Endereços". Em vez de o app cliente fazer 3 chamadas, ele faz uma só para o Gateway (ex: `GET /meu-perfil`). O Gateway, então, chama os 3 serviços internos, junta tudo (agrega) e devolve uma resposta única.

Gabarito: C

14.(PROF. VINICIUS BORGES / INÉDITA - 2025)

Em um API Gateway, a implementação de cache de respostas é uma estratégia utilizada primariamente para:

- A) Aumentar a segurança, validando o token de autenticação.
- B) Reduzir a latência para o cliente e diminuir a carga nos microserviços de backend.
- C) Garantir que o cliente não exceda o limite de requisições por segundo.
- D) Traduzir o formato da mensagem entre o cliente e o microserviço.

Comentários:

Por que fazemos cache? Para duas coisas: 1) **Entregar a resposta mais rápido** para o cliente (reduzir latência), já que a resposta está pronta no Gateway e não precisamos ir até o microserviço. 2) **Proteger o microserviço de carga desnecessária** (diminuir a carga no backend). Se 1000 pessoas pedem a "lista de produtos" em um segundo, o Gateway busca no microserviço uma vez e responde as outras 999 com a cópia em cache.

Gabarito: B

GABARITO

GABARITO



- 1. D
- 2. A
- 3. E
- 4. C
- 5. A

- 6. C
- 7. E
- 8. E
- 9. C
- 10. D

- 11. B
- 12. E
- 13. C
- 14. B

LISTA DE QUESTÕES

1. (Ano: 2024 Banca: IMPARH Órgão: Prefeitura de Fortaleza - CE Prova: IMPARH - 2024 - Prefeitura de Fortaleza - CE - Analista de Regulação - Ciências da Computação)

Qual componente é descrito pela seguinte definição?

"Um padrão de software amplamente utilizado em arquiteturas de microsserviços, atuando como um ponto de entrada único para todas as requisições dos clientes e roteando essas requisições para os microsserviços apropriados."

- A) Reverse Proxy.
- B) Load Balancer.
- C) Service Mesh.
- D) API Gateway.

2. (Ano: 2024 Banca: COMPERVE - UFRN Órgão: UFRN Prova: COMPERVE - UFRN - 2024 - UFRN - Técnico em Tecnologia da Informação (Suporte e Redes))

O serviço atua como um intermediário entre os clientes externos e os servidores internos, recebendo as requisições dos clientes e as encaminhando para os servidores de fato. Com esse serviço, é possível ocultar a infraestrutura do servidor, oferecer armazenamento em cache, aprimorar a segurança e possibilitar um controle mais granular sobre as requisições.

Esse serviço descrito é o:

- A) Proxy reverso.
- B) Load balancer.
- C) API Gateway.
- D) NAT - Network Address Translation.

3. (Ano: 2024 Banca: CESPE / CEBRASPE Órgão: CAU-BR Prova: CESPE / CEBRASPE - 2024 - CAU-BR - Analista de Infraestrutura de Tecnologia da Informação)

Com relação a barramento de serviços corporativos (EBS), e dos aspectos de segurança e versatilidade do API Gateway, julgue o item subsequente.

Uma das vantagens de se usar um API Gateway é que ele permite implementar a segurança da API em um único ponto, em vez de em cada serviço individual.

4. (Ano: 2023 Banca: CESPE / CEBRASPE Órgão: SEFIN de Fortaleza - CE Prova: CESPE / CEBRASPE - 2023 - SEFIN de Fortaleza - CE - Analista Fazendário Municipal - Área de Conhecimento: Ciência da Computação, Informática/Processamento de Dados)

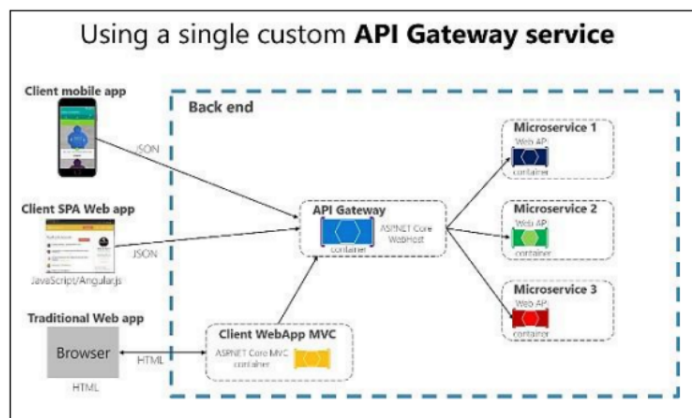
Acerca de API Gateway, julgue o próximo item.

Considere-se que seja necessário criar uma API Gateway para um serviço back-end que responde a solicitações HTTP na rota /api/fiscalização. Nessa situação hipotética, na etapa de configuração da rota para o serviço de back-end, é necessário especificar, na API Gateway, o método HTTP (GET, POST etc.) que a aplicação back-end suporta, para que a integração funcione corretamente.

5. (Ano: 2024 Banca: Instituto Access Órgão: Banestes Prova: Instituto Access - 2024 - Banestes - Analista em Tecnologia da Informação - Segurança da Informação)

A Figura mostra como um Gateway de API personalizado pode se encaixar em uma arquitetura simplificada baseada em microsserviço com apenas alguns microsserviços. Um Gateway de API pode oferecer vários recursos. Dependendo do produto, ele pode oferecer recursos mais avançados ou mais simples, no entanto, os recursos.

Marque a alternativa que apresenta corretamente, uma das mais importantes e fundamentais para qualquer Gateway de API padrões de design.



- A) Proxy reverso ou roteamento de gateway.
- B) Porta TCP diferente para cada microsserviço.
- C) Balanceador de carga e ADC.
- D) Hosts descarregando terminação SSL.

6. (Ano: 2024 Banca: CESPE / CEBRASPE Órgão: SEPLAG-CE Prova: CESPE / CEBRASPE - 2024 - SEPLAG-CE - Analista de Gestão Pública - Área de Atuação: Ciência da Computação)

Acerca de gerenciamento de API, de RESTful e de ITIL 4, julgue o item subsequente.

Um API gateway atua como um ponto central de entrada para várias APIs e desempenha um papel importante na simplificação da gestão de tráfego, autenticação, autorização e monitoramento das chamadas de API.

7. (Ano: 2025 Banca: CESPE / CEBRASPE Órgão: MPE-CE Prova: CESPE / CEBRASPE - 2025 - MPE-CE - Analista Ministerial - Especialidade: Ciências da Computação)

Julgue o item a seguir, referente a ferramentas de integração assíncrona, microsserviços, arquitetura orientada a serviços e gerenciamento de serviços.

Um API gateway é responsável pelo armazenamento de dados temporários, e, apesar de executar várias tarefas transversais, não realiza algumas funções, tais como terminação de SSL, TLS mútuo e limitação de taxa.

8. (Ano: 2024 Banca: CESPE / CEBRASPE Órgão: MPO Prova: CESPE / CEBRASPE - 2024 - MPO - Analista de Planejamento e Orçamento - Especialidade: Desenvolvimento de Sistemas Orçamentários)

A respeito de arquitetura de aplicações, julgue o próximo item.

Um gateway de API funciona como um portfólio de serviços, em que o usuário escolhe e implementa a chamada do serviço necessário.

9. (Ano: 2025 Banca: FGV Órgão: DPE-RO Prova: FGV - 2025 - DPE-RO - Analista Programador - Classe B, adaptada)

Um *API Gateway* atua como um *proxy* reverso, ficando entre os aplicativos cliente e os microsserviços, fornecendo um único ponto de entrada para todas as requisições de clientes a um conjunto de serviços.

10. (Ano: 2025 Banca: FCC Órgão: TRT - 6ª Região (PE) Prova: FCC - 2025 - TRT - 6ª Região (PE) - Analista Judiciário - Área Apoio Especializado - Especialidade: Tecnologia da Informação)

Ao usar o Gateway API com o Service Discovery no contexto da PDPJ-Br, a principal consequência para o frontend em uma aplicação baseada em microsserviços é que o

- A) Gateway API diminui a necessidade de autenticação, uma vez que o Service Discovery é integrado ao SSO.
- B) frontend deve configurar manualmente os IPs e portas dos serviços disponíveis na plataforma.
- C) frontend se torna o responsável pela chamada direta de cada microsserviço individualmente.
- D) Gateway API reduz o número de chamadas externas que o frontend precisa fazer, tornando a comunicação mais eficiente.
- E) frontend deve gerenciar manualmente as atualizações de endpoint de cada serviço individual registrado.

11. (PROF. VINICIUS BORGES / INÉDITA - 2025)

Em arquiteturas de microsserviços, é comum que diferentes clientes (ex: um app mobile e uma aplicação web desktop) tenham necessidades distintas de consumo de dados. O padrão de API Gateway que endereça essa necessidade, provendo endpoints otimizados para cada tipo de cliente, é conhecido como:

- A) Gateway Aggregation
- B) Backend for Frontends (BFF)
- C) Service Mesh
- D) Proxy Reverso Adaptativo
- E) Serverless

12. (PROF. VINICIUS BORGES / INÉDITA - 2025)

Um API Gateway e um Service Mesh são componentes que desempenham funções idênticas, sendo responsáveis primariamente pela comunicação Leste-Oeste (interna, entre microserviços).

13. (PROF. VINICIUS BORGES / INÉDITA - 2025)

O padrão *Gateway Aggregation* refere-se à capacidade do API Gateway de receber uma única requisição do cliente e, internamente, orquestrar chamadas a múltiplos microserviços, consolidando as respostas antes de retorná-las ao cliente.

14. (PROF. VINICIUS BORGES / INÉDITA - 2025)

Em um API Gateway, a implementação de cache de respostas é uma estratégia utilizada primariamente para:

- A) Aumentar a segurança, validando o token de autenticação.
- B) Reduzir a latência para o cliente e diminuir a carga nos microserviços de backend.
- C) Garantir que o cliente não exceda o limite de requisições por segundo.
- D) Traduzir o formato da mensagem entre o cliente e o microserviço.

GABARITO

GABARITO



- 1. D
- 2. A
- 3. E
- 4. C
- 5. A

- 6. C
- 7. E
- 8. E
- 9. C
- 10. D

- 11. B
- 12. E
- 13. C
- 14. B

ESSA LEI TODO MUNDO CONHECE: PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.



1 Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.



2 Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).



3 Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).



4 Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).



5 Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).



6 Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).



7 Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.



8 O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.



Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.